

REPUBLIQUE DU CAMEROUN
Paix-Travail-Patrie

MINISTÈRE DE L'ENSEIGNEMENT
SUPÉRIEURE

UNIVERSITE DE YAOUNDE I
ÉCOLE NATIONALE SUPÉRIEURE
POLYTECHNIQUE DE YAOUNDE

DÉPARTEMENT DU GÉNIE ÉLECTRIQUE ET
TÉLÉCOMMUNICATIONS



REPUBLIC OF CAMEROON
Peace-Work-Fatherland

MINISTRY OF HIGHER
EDUCATION

UNIVERCITY OF YAOUNDE I
NATIONAL ADVANCED SCHOOL
POLYTECHNIQUE OF YAOUNDE

DEPARTEMENT OF GENIE ELECTRIQUE
AND TELECOMMUNICATION

RAPPORT DE GENIE LOGICIEL

**THEME : CONCEPTION D'UNE APLICATION WEB POUR LA
GESTION D'UN RESTAURANT**

Rédigé Par :

NOMS ET PRENOMS	MATRICULE	PARTICIPATION (%)
ETOLO NDONGO EMARAN LOUIS PHILIPPE	23P268	30
DJOUMA NGONGANG NOE BRADDY	22P386	30
MEKO'O EMVOLO ANGELA CLAUDE	25P883	30
BOMO AKAM ROLAND	20P416	10
YOUWE LAMNA SYLVAIN	21P388	0

Groupe 8 Sous l'encadrement de : Mr AMOS

ANNÉE ACADÉMIQUE

2025/2026

SOMMAIRE

SOMMAIRE	2
LISTE DES FIGURES.....	4
LISTE DES TABLEAUX.....	6
INTRODUCTION.....	7
Phases du projet.....	7
Phase I: préparation du projet et initiation aux setup	8
Phase II : développement du système	8
Phase III: test et integration.....	8
Partie I : installation de l'environnement de développement	8
Partie II : Analyse et développement du système	12
PARTIE III : IMPLÉMENTATION DES MODULES DE L'APPLICATION	47
PARTIE IV : BIBLIOTHÈQUES UTILISÉES	56
PARTIE V : TESTS ET VALIDATION	59
CONCLUSION	60
WEBOGRAPHIE.....	61
TABLE DE MATIRE	62

LISTE DES FIGURES

Figure 1: bugs d'installation de Django.....	9
Figure 2: diagramme de cas d'utilisation.....	15
Figure 3: diagramme de classe	16
Figure 4: Inscription et Authentification du client (D.S)	18
Figure 5: Passage d'une commande en ligne du client(D.S)	19
Figure 6: Réservation d'une table (D.S)	20
Figure 7: Suivi de commande en temps réel (D.S client).....	21
Figure 8: Téléchargement de la facture(D.S client)	22
Figure 9: Création d'une commande sur place (D.S serveur).....	23
Figure 10: Générer facture (D.S serveur).....	24
Figure 11: Gérer réservation(D.S serveur)	25
Figure 12:Gestion de la fiche client (D.S serveur).....	26
Figure 13: Authentication sécurisée(D.S admin)	27
Figure 14: Gestion des utilisateurs et des rôles(D.S admin)	28
Figure 15: Gestion des Ressources Humaines (RH) et des salaires (D.S admin)	29
Figure 16: Configuration globale du système (D.S admin)	30
Figure 17: création d'une recette (D.S chef).....	31

Figure 18: modification d'une recette (D.S chef)	33
Figure 19: supervision de la cuisine (D.S chef)	34
Figure 20: Préparation et validation d'une commande(D.S cuis).....	35
Figure 21: Signalement d'une rupture de stock (D.S cuis).....	36
Figure 22: Consultation et mise à jour du stock (D.S gest).....	38
Figure 23: Passage de commande aux fournisseurs (D.S gest).....	39
Figure 24: Assignation des livraisons clients 'D.S gest)	40
Figure 25: Prise en charge d'une livraison	42
Figure 26: Confirmation de la livraison(D.S livr).....	43
Figure 27: Diagramme de déploiement	44

LISTE DES TABLEAUX

Tableau 1: Phases de développement.....	7
Tableau 2: Méthodes de développement.....	12
Tableau 3: explication du diagramme de cas d'utilisation	14
Tableau 4: Synthèse des Relations et Dépendances.....	14
Tableau 5: récapitulatif des 22 tables reparties en modules du diagramme de classe.....	16
Tableau 6: Répartition des taches.....	45
Tableau 7: Module de gestion des utilisateurs	48
Tableau 8 :difficultés module de gestion des recettes	49
Tableau 9: difficultés module de gestion des produits et du catalogue.....	50
Tableau 10: Difficultés module de gestion des produits et du catalogue.....	51
Tableau 11: Difficultés module de gestion des ressources humaines (rh)	52
Tableau 12: Difficultés module tableau de bord et rapports financiers	53
Tableau 13: Difficultés rencontrées lors des tests	59

INTRODUCTION

Le Génie logiciel est une discipline de l'ingénierie qui permet de concevoir, développer et maintenir les logiciels à l'aide des méthodes structurées. Dans le cadre de ce cours, nous avons été amenés à réaliser un projet concret mettant en application les principes fondamentaux du Génie Logiciel. Notre équipe, composée de cinq membres, a choisi de se pencher sur une problématique courante mais complexe : la digitalisation des services de restauration.

Conscients des défis liés à la gestion manuelle des commandes, lenteur du service, erreurs de facturation et manque de visibilité sur les stocks, nous avons uni nos compétences pour concevoir et développer une application web de gestion de restaurant baptisée (Fast-food)

Ce projet ne représente pas seulement pour nous un exercice académique, mais une véritable immersion dans le cycle de vie d'un logiciel. De l'analyse rigoureuse des besoins à la modélisation UML, jusqu'au développement technique du Frontend et du Backend, nous avons structuré notre démarche pour répondre aux exigences de performance et de sécurité du secteur.

Dans la suite de ce rapport, nous détaillons notre méthodologie de conception, les choix technologiques que nous avons opérés ainsi que les différentes étapes qui nous ont permis de transformer un besoin fonctionnel en une solution numérique opérationnelle.

Phases du projet

Notre projet sera réparti en trois phases chacune ayant un objectif spécifique. Les phases de ce projet sont les suivantes :

Tableau 1: Phases de développement

Phase	Livrable
Phase I	Cahier des charges, Environnement de développement prêt.
Phase II	Code source (Front & Back), Base de données fonctionnelle, Diagrammes UML
Phase III	Rapport de tests, Application déployée et fonctionnelle.

Phase I: préparation du projet et initiation aux setup

Cette phase se concentre sur la compréhension des exigences du projet et la préparation de l'environnement de développement. Elle comprend l'installation des outils nécessaires, la création de la structure du projet **Django**, l'analyse de la base de données SQL et la connexion de l'application à la base de données.

Phase II : développement du système

Cette phase est dédiée à la construction des fonctionnalités principales de l'application. Elle inclut le développement de la logique backend à l'aide de **Django**, la conception de l'interface utilisateur et la mise en œuvre des modules de base tels que la gestion des clients, les commandes, les réservations, les employés et l'inventaire.

Phase III: test et integration

Cette phase vise à garantir la fiabilité de l'application et sa préparation au déploiement. Elle comprend la réalisation de tests unitaires sur les modules individuels, l'exécution de tests d'intégration pour vérifier la communication entre le backend Django et le frontend, ainsi que les tests de bout en bout du flux complet de gestion du restaurant. Enfin, elle inclut la correction des bugs et la validation de la conformité du système aux exigences initiales du projet.

PARTIE I : INSTALLATION DE L'ENVIRONNEMENT DE DÉVELOPPEMENT

1.1 INSTALLATION DE DJANGO

Django est un Framework web basé sur Python. Il suit l'architecture MVT (Modèle-Vue-Template) et permet de développer des sites sécurisés et robustes rapidement. Django gère automatiquement de nombreuses tâches complexes : la connexion à la base de données, la gestion des URL, l'authentification des utilisateurs et la protection contre les attaques courantes.

Pour l'installation de Django nous avons utilisé deux méthodes à savoir :

- L'invite de commande Windows : via la commande ' pip Install Django' difficulté rencontrée : l'invite de commande + lance l'installation de Django mais l'installation n'arrive pas à terme

```
C:\Sélection Inverse de commandes>
Microsoft Windows [version 10.0.22631.5624]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\ALMATA>pip django
ERROR: unknown command "django"

C:\Users\ALMATA>pip install django
Collecting django
  Downloading django-6.0.3-py3-none-any.whl (8.4 MB)
    -----
      2.1/8.4 MB 7.9 kB/s eta 0:13:14
ERROR: Exception:
Traceback (most recent call last):
  File "C:\Users\ALMATA\AppData\Local\Programs\Python\Python312\Lib\site-packages\pip\_vendor\urllib3\response.py", line 438, in _error_catcher
    yield
  File "C:\Users\ALMATA\AppData\Local\Programs\Python\Python312\Lib\site-packages\pip\_vendor\urllib3\response.py", line 561, in read
    data = self._fp_read(amt) if not fp_closed else b""
           ^^^^^^^^^^^^^^
  File "C:\Users\ALMATA\AppData\Local\Programs\Python\Python312\Lib\site-packages\pip\_vendor\urllib3\response.py", line 527, in _fp_read
    return self._fp.read(amt) if amt is not None else self._fp.read()
           ^^^^^^^^^^^^^^
  File "C:\Users\ALMATA\AppData\Local\Programs\Python\Python312\Lib\site-packages\pip\_vendor\cachecontrol\filewrapper.py", line 90, in read
    data = self._fp.read(amt)
           ^^^^^^^^^^^^^^
  File "C:\Users\ALMATA\AppData\Local\Programs\Python\Python312\Lib\http\client.py", line 472, in read
    s = self.fp.read(amt)
        ^^^^^^^^^^^^^^
  File "C:\Users\ALMATA\AppData\Local\Programs\Python\Python312\Lib\socket.py", line 707, in readinto
    return self._sock.recv_into(b)
           ^^^^^^^^^^^^^^
  File "C:\Users\ALMATA\AppData\Local\Programs\Python\Python312\Lib\ssl.py", line 1216, in recv_into
    return self.read(nbytes, buffer)
           ^^^^^^^^^^^^^^
  File "C:\Users\ALMATA\AppData\Local\Programs\Python\Python312\Lib\ssl.py", line 1072, in read
    return self._sslobj.read(len, buffer)
           ^^^^^^^^^^^^^^
TimeoutError: The read operation timed out

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
```

Figure 1: bugs d'installation de Django

Après de nombreuses tentatives nous n'avons pas pu résoudre le problème nous avons terminé notre installation sur le terminal Vscode via la même commande

Vérification et création d'un projet Django :

```
django-admin --version  
django-admin startproject nom_du_projet  
cd nom_du_projet  
python manage.py runserver
```

La commande **runserver** lance un serveur local accessible à l'adresse <http://127.0.0.1:8000>

1.2. INSTALLATION DE PYTHON

Python est un langage de programmation interprété, polyvalent et facile à apprendre. Il est aujourd'hui l'un des langages les plus utilisés dans le monde, notamment pour le développement web (backend), l'automatisation de tâches, la science des données et l'intelligence artificielle. Dans notre contexte, Python est la base indispensable sur laquelle repose Django. Python utilise un interpréteur qui exécute le code ligne par ligne, ce qui facilite le débogage et l'apprentissage. Lors de l'installation sous Windows, il est impératif de cocher la case "**Add Python to PATH**" pour que le système reconnaisse les commandes Python dans le terminal. pip est le gestionnaire de paquets intégré permettant d'installer des bibliothèques supplémentaires.

Vérification de l'installation :

```
Python3 --version
```

```
pip3 --version
```

1.3. INSTALLATION DE VISUAL STUDIO CODE (VS CODE)

Visual Studio Code est un éditeur de code source développé par Microsoft. Léger mais puissant, il supporte une multitude de langages de programmation et offre un écosystème riche d'extensions. C'est l'environnement de développement intégré (IDE) que nous utilisons pour écrire, tester et déboguer notre code Python et Django. Il a des fonctionnalités importantes comme la Coloration syntaxique, autocomplétion intelligente (IntelliSense), débogueur intégré. Il bénéficie aussi d'une intégration native avec Git pour gérer les versions du code directement depuis l'éditeur.

Après l'installation, nous avons ajouté l'extension "**Python**" de **Microsoft** pour bénéficier de l'autocomplétion et du débogage, "**Django**"

1.4. INSTALLATION DE GIT ET GITHUB

Git est un système de contrôle de versions décentralisé. Il permet de suivre l'historique de toutes les modifications apportées au code d'un projet. GitHub est une plateforme en ligne qui héberge des dépôts Git et facilite la collaboration entre développeurs. Il gère les versions du code c'est-à-dire qu'on peut revenir à n'importe quelle version précédente du code en cas d'erreur. C'est également une application de Collaboration où Plusieurs développeurs peuvent travailler en parallèle sur les mêmes fichiers grâce aux branches. GitHub permet de sauvegarder le code en ligne et de le partager facilement.

Installation de Git :

```
winget install --id Git.Git -e --source winget
```

Configuration initiale (obligatoire après l'installation) :

```
git config --global user.name "Votre Nom"
```

```
Git config --global user.email "votre.email@example.com"
```

Commandes fondamentales Git :

```
git init                # Initialiser un dépôt local
git clone <url_du_depot> # Cloner un dépôt GitHub
git add .               # Ajouter tous les fichiers modifiés
git commit -m "message" # Enregistrer les modifications
git push origin main    # Envoyer les modifications sur GitHub
git pull               # Récupérer les dernières modifications
```

1.5. STARUML

StarUML est un outil de modélisation logicielle basé sur la notation UML (Unified Modeling Language). Il permet de concevoir l'architecture d'un système avant de commencer à coder. En informatique, modéliser un système avant de le développer est une bonne pratique qui évite les

erreurs de conception coûteuses à corriger plus tard. Il supporte les Diagrammes de classes, de séquence, de cas d'utilisation (use case), d'activité, etc. Il permet de visualiser les relations entre les différentes entités du projet. On peut exporter des diagrammes en image (PNG, SVG) pour les intégrer dans des rapports.

StarUML s'installe via son site officiel (staruml.io).

PARTIE II : ANALYSE ET DÉVELOPPEMENT DU SYSTÈME

2.1 ANALYSE DES MÉTHODES EMPLOYÉES

La phase d'analyse de méthode consiste à décrire toutes les actions et les méthodes employées en vue de la réalisation de notre projet. En ce sens, nous présentons ici deux méthodes de réalisation d'un projet à travers le tableau ci-dessous :

Tableau 2: Méthodes de développement

Méthodes	Avantages	Inconvénients
Méthodes Traditionnelles ou classiques	- L'intégralité du projet est définie et planifiée avant le lancement du projet ; -Le budget est connu dès le départ.	- Difficulté de revenir sur une étape précédente ; -Les exigences des clients ne sont pas prises en compte ; -Risque d'effet tunnel.
Méthodes agiles	-Adaptabilité et flexibilité ; -Livraison plus fréquente ; -Qualité contrôlée plus régulièrement par le client ; -Satisfaction du client.	-Le projet peut avancer lentement en raison d'une trop grande flexibilité ; -Risque de dépassement des coûts et des délais si le client change d'avis.

Les méthodes traditionnelles sont également utilisées pour des projets ne nécessitant pas de documentation détaillée.

Nous avons choisi la **méthode agile** principalement pour sa capacité à supporter un **développement module par module** (approche incrémentale), ce qui est essentiel pour sécuriser la conception de notre base de données MySQL. Cette stratégie nous permet de construire, tester et valider chaque brique fonctionnelle de manière isolée qu'il s'agisse de la gestion des commandes ou des stocks garantissant ainsi une intégration fluide et sans erreur du système global. En progressant par itérations successives, nous assurons une livraison continue de valeur tout en restant capables d'ajuster les fonctionnalités aux besoins réels du restaurant.

2.2. ANALYSE ET MODÉLISATION UML

La modélisation est une représentation graphique des modèles utilisés dans l'application. Elle répond à la question de savoir comment les éléments métiers ont été modélisés dans notre application. Nous avons le choix entre les outils de modélisation Merise et UML (Unified Modeling Language).

Le premier outil (Merise) renvoie à la méthodologie de modélisation à usage général dans le domaine du développement des systèmes d'information et du génie logiciel. Cette méthode est tournée vers la compréhension et la formalisation des besoins métier.

Nous avons opté pour le second (UML) qui est un langage de modélisation des systèmes standards qui utilise des diagrammes pour représenter chaque aspect de notre système sur la base du concept de l'orienté objet.

2.2.1. DIAGRAMME DE CAS D'UTILISATION

Le diagramme de cas d'utilisation est une représentation fonctionnelle d'un système. Il présente une unité discrète d'interaction entre un utilisateur et un système.

Tableau 3: explication du diagramme de cas d'utilisation

Acteur	Rôles & Missions Principales
Client	S'authentifier, consulter le menu, passer commande (en ligne), réserver/annuler, suivre commande, télécharger facture.
Serveur	Créer commandes (sur place), générer factures, gérer les réservations, gérer les fiches clients.
Administrateur	Authentification sécurisée, gestion RH/salaires, gestion des rôles, tableau de bord, configuration système.
Chef Cuisinier	Création/modification de recettes, supervision des équipes, consultation des commandes en cours.
Cuisinier	Exécution des commandes, consultation recettes, validation plat prêt, signalement de rupture.
Livreur	Prise en charge des livraisons, confirmation de remise, signalement de problèmes de trajet.
Gestionnaire de Stock	Contrôle inventaire (alertes seuil), gestion fournisseurs, approvisionnements, assignation des livraisons.

Tableau 4: Synthèse des Relations et Dépendances

Type de Relation	Description des Flux
« include » (Obligatoire)	La création d'une commande inclut le calcul du montant. Toute commande en livraison inclut la création d'une livraison.
« extend » (Optionnel)	Le signalement de rupture étend la mise à jour des stocks. Un problème de livraison étend la confirmation de livraison.
Partage de Cas	Le Client et le Serveur partagent la consultation du catalogue et la gestion des réservations. Le Livreur et le Gestionnaire partagent la gestion des déplacements.

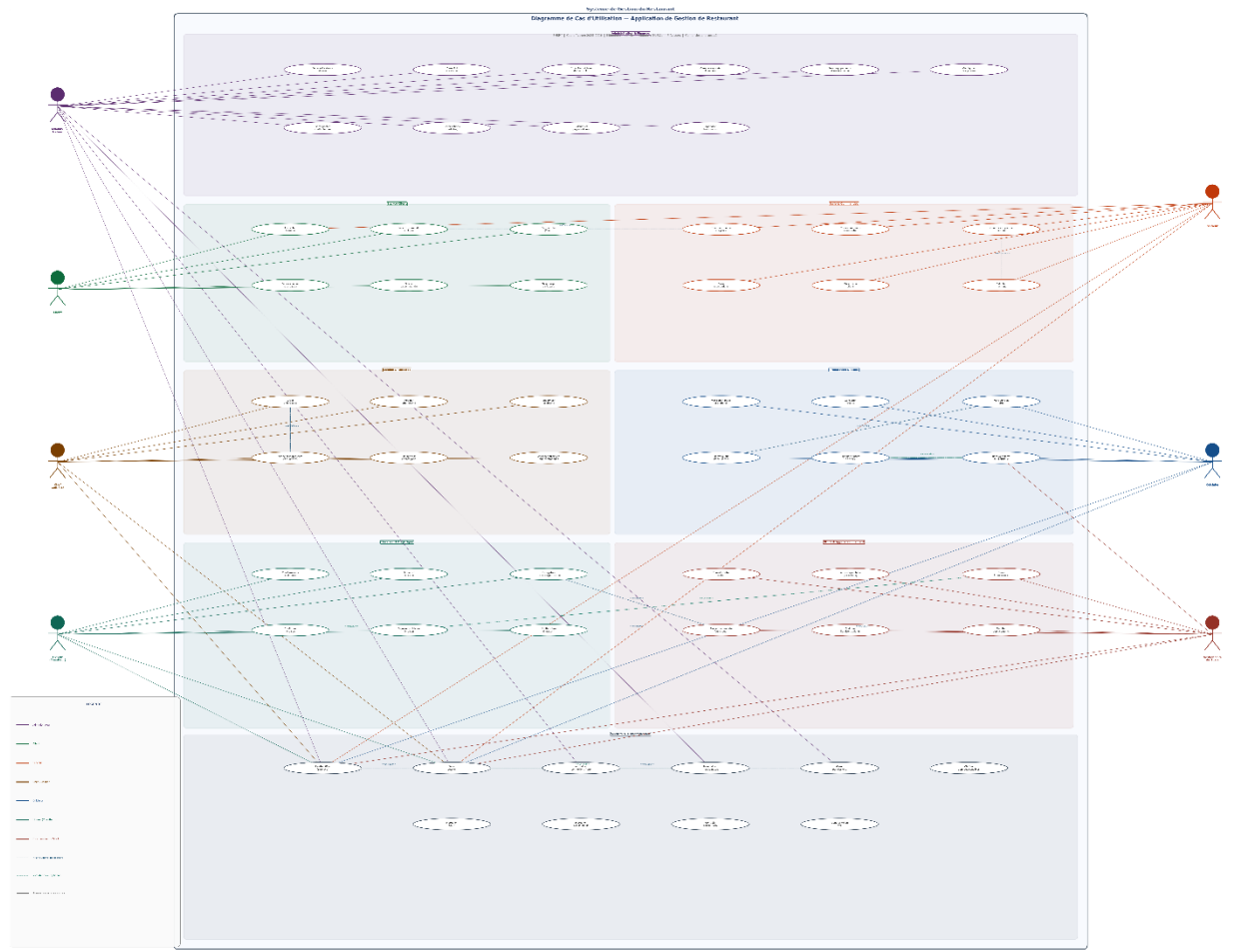


Figure 2: diagramme de cas d'utilisation

2.2.2. DIAGRAMME DE CLASSE

Le diagramme de classe est le pilier central de la modélisation orientée objet en UML (Unified Modeling Language), offrant une représentation statique et structurelle d'un système. Il décrit les classes de données, leurs attributs, leurs méthodes ainsi que les relations (héritage, agrégation, composition, etc.) qui les lient. Ses objectifs principaux sont de fournir une vue d'ensemble de l'architecture logicielle, de faciliter la communication entre les développeurs en définissant un vocabulaire commun, et de servir de base à la génération automatique de code, tout en assurant la cohérence entre les besoins fonctionnels et la structure technique.

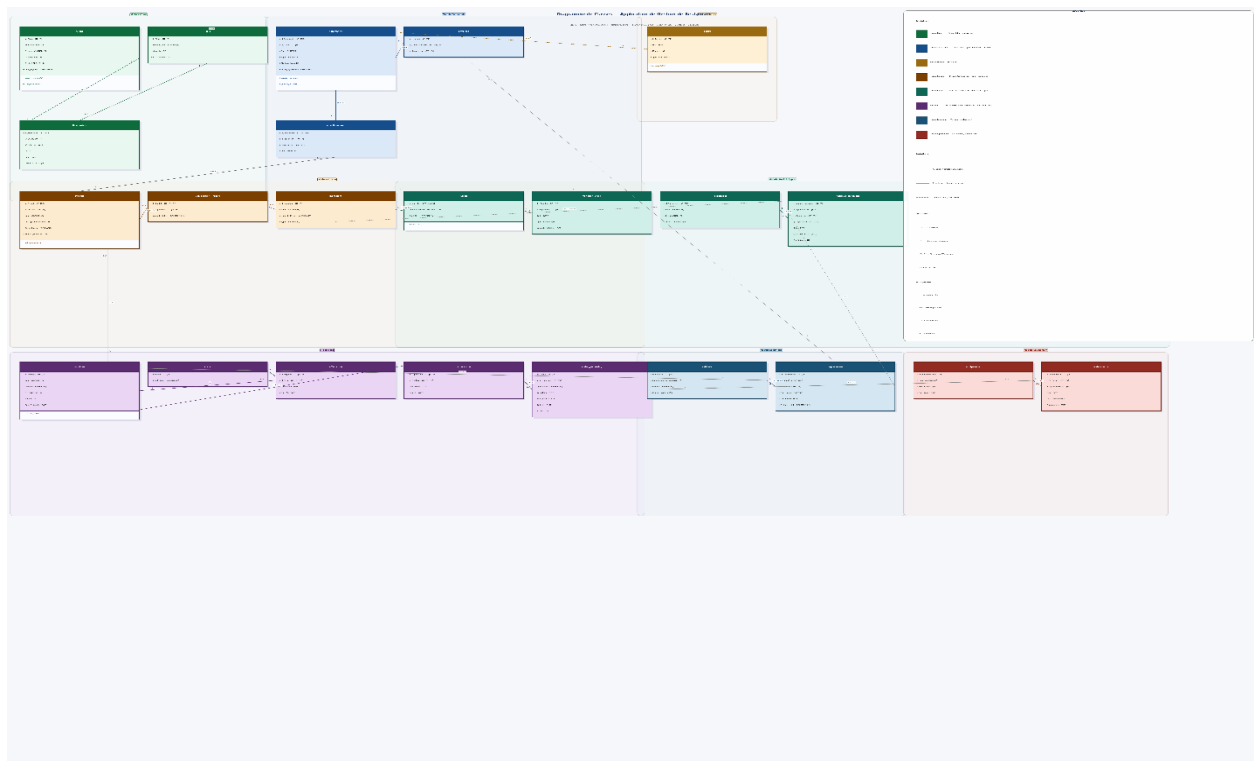


Figure 3: diagramme de classe

Nous avons regroupé les 22 tables de la base de donnée en 8 modules consignés dans le tableau suivant

Tableau 5: récapitulatif des 22 tables reparties en modules du diagramme de classe

Module	Classes (Tables) associées
Client & Réservation	Client, Table, Reservation
Ventes & Commandes	Commande, Ligne_Commande, Livraison
Gestion Financière	Facture
Cuisine & Recettes	Produit, Composition Produit, Ingredient
Stock & Achats	Stock, Variation_Stock, Fournisseur, Approvisionnement
RH & Marketing	Employe, Poste, Affectation, Supervision, Action_Marketing
Logistique	Vehicule, Deplacement
Maintenance	Equipement, Maintenance

2.2.3. DIAGRAMME DE SÉQUENCE

Le diagramme de séquence est un outil de modélisation comportemental d'UML qui représente l'ordre chronologique des interactions entre les différents objets d'un système pour accomplir une fonctionnalité précise. Il met en scène des acteurs et des objets le long de lignes de vie verticales, où les messages échangés sont représentés par des flèches horizontales classées de haut en bas selon le temps. Ses objectifs principaux sont de détailler la logique des scénarios d'utilisation (cas d'utilisation), de mettre en évidence la chronologie des opérations complexes et de clarifier la répartition des responsabilités entre les composants du système, facilitant ainsi la conception des interfaces de programmation et le débogage des flux de données.

Nous avons pour chaque utilisateur et en fonction de chaque scénario fait les diagrammes de séquences

- **Client**

Le Client est l'utilisateur final pour qui l'expérience doit être la plus simple possible. Son interface lui permet d'accéder aux services du restaurant (repas et réservations) de manière autonome et sécurisée.

- **Scenario 1 : Inscription et Authentification**

- * Le client remplit ses informations personnelles (nom, email, mot de passe) ou se connecte à son compte existant.
- * Le système contrôle l'unicité de l'email et la validité du mot de passe dans la base de données.
- * En cas de succès, il ouvre une session sécurisée qui permettra au client de retrouver son historique et ses factures.

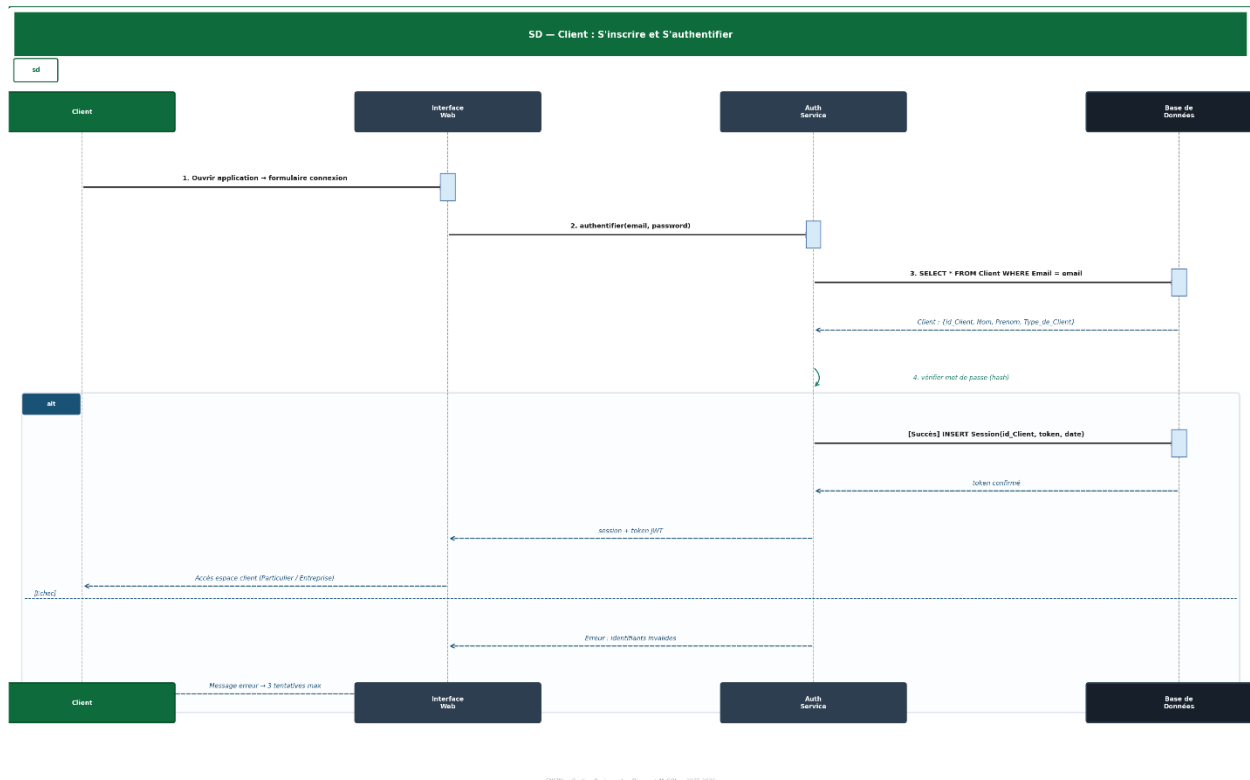


Figure 4: Inscription et Authentification du client (D.S)

- **Scenario 2 : Passage d'une commande en ligne**

- * Le client parcourt le catalogue de plats, les ajoute à son panier et valide son choix pour une livraison à domicile.
- * Le système vérifie instantanément si les ingrédients sont disponibles en stock pour chaque plat choisi.
- * Il génère simultanément la commande, le calcul automatique du prix et planifie une nouvelle livraison.
- * Le système affiche un récapitulatif et confirme que la commande est envoyée en cuisine.

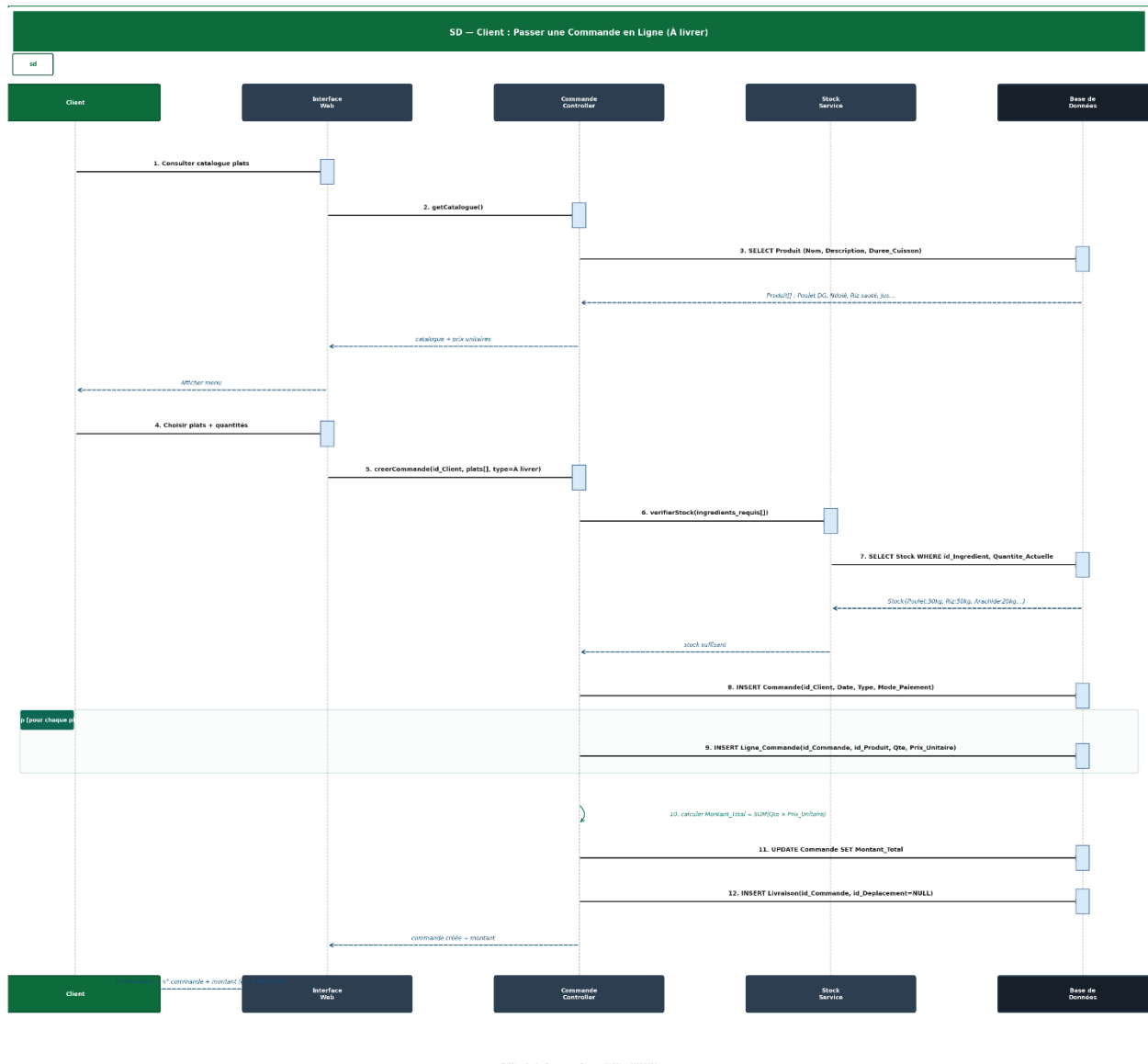


Figure 5: Passage d'une commande en ligne du client(D.S)

- **Scenario 3 : Réservation d'une table**

- * Il choisit une date, une heure et le nombre de personnes pour venir manger sur place.
- * Le système croise les données des tables physiques et des réservations déjà enregistrées.

- * Si une table est libre, il bloque le créneau et marque la réservation comme "Confirmée" dans l'agenda du restaurant.

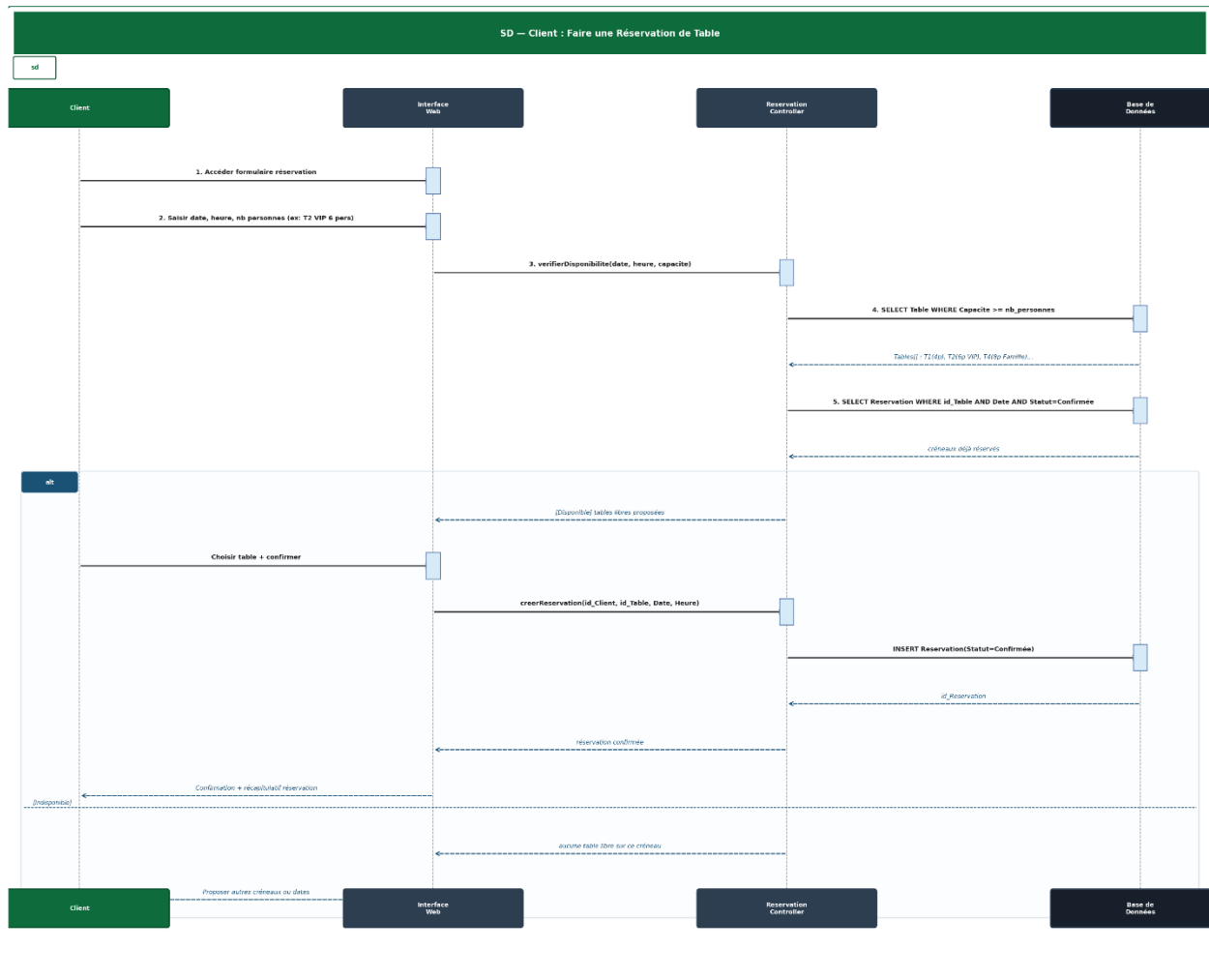


Figure 6: Réservation d'une table (D.S)

• Scenario 4 : Suivi de commande en temps réel

- * Il consulte son espace personnel pour savoir où en est son repas.
- * Le système interroge les informations venant du cuisinier (préparation) et du livreur (déplacement).
- * Il retourne le statut précis au client (ex : "En préparation", "En cours de livraison" ou "Livré").

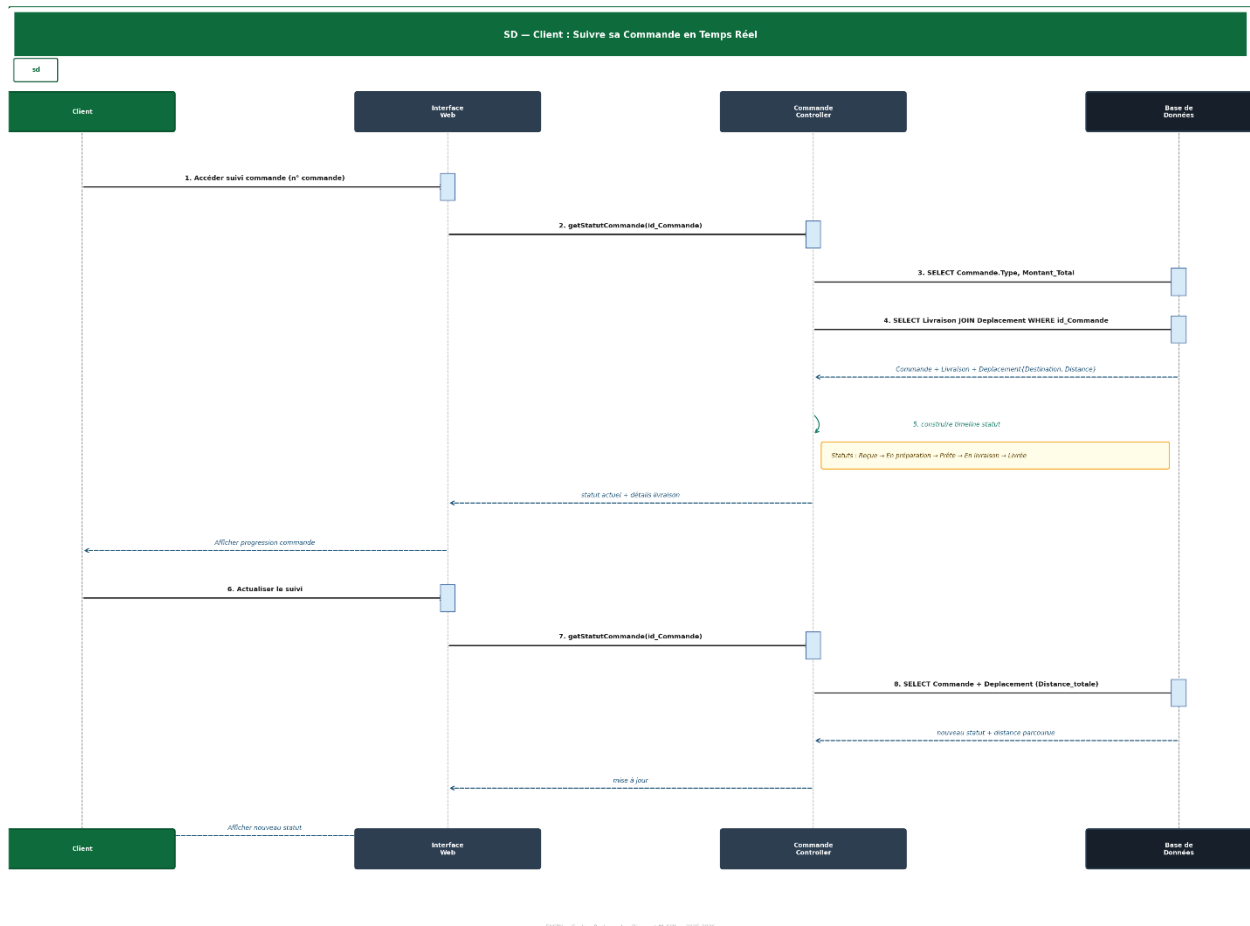


Figure 7: Suivi de commande en temps réel (D.S client)

• **Scenario 5 : Téléchargement de la facture**

- * Il clique sur l'option pour obtenir son justificatif de paiement
- * Le système extrait les lignes de commande et les tarifs pour créer instantanément un fichier PDF.
- * Il rend le fichier disponible au téléchargement immédiat dans l'interface du client.

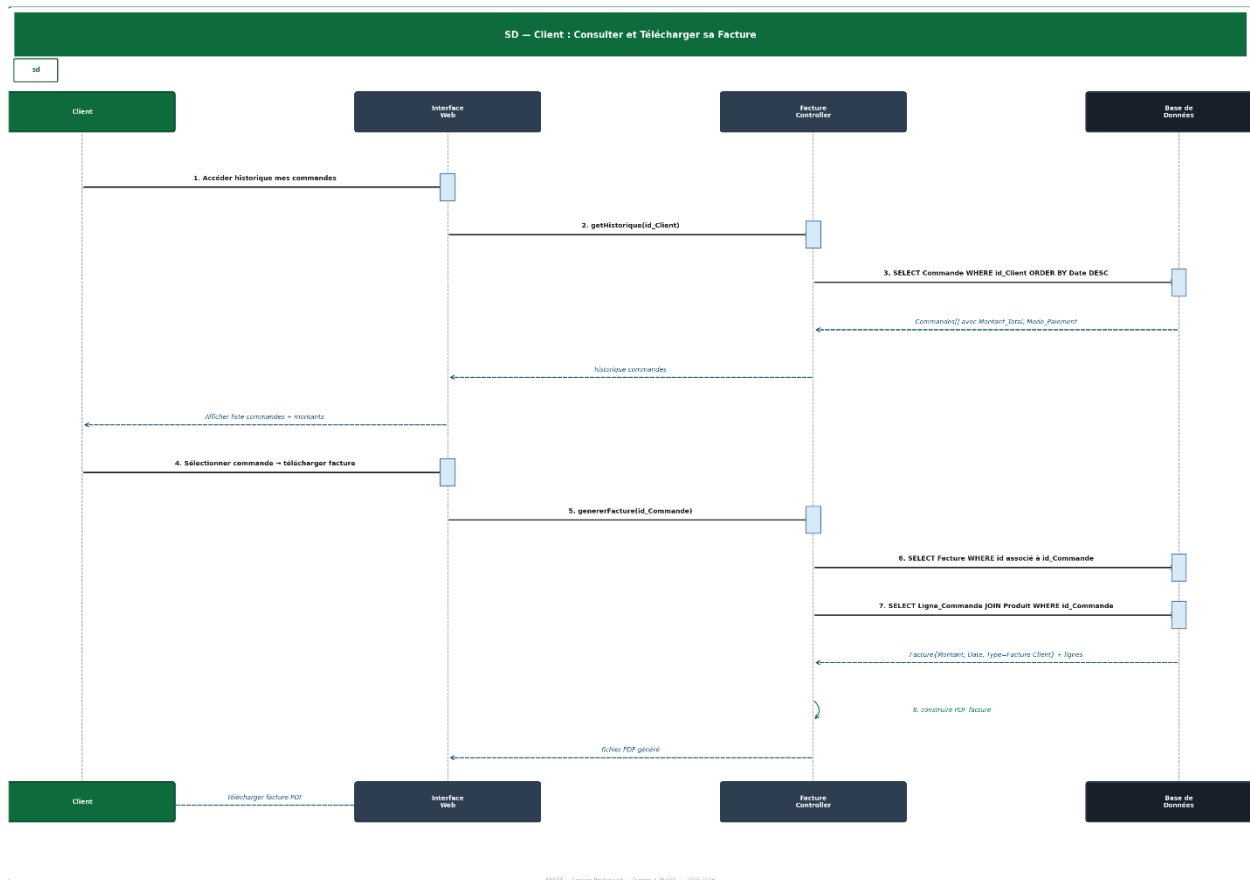


Figure 8: Téléchargement de la facture(D.S client)

- SERVEUR

Le serveur assure le lien direct entre le client et la cuisine. Son rôle est centré sur la prise en charge des besoins immédiats des clients et le suivi des commandes

• **Scenario 1 : Création d'une commande sur place**

- * Le serveur sélectionne le numéro de la table et choisit les différents plats demandés par le client dans le menu numérique.
- * Il crée une nouvelle commande de type "Sur place" dans la base de données.
- * Le système additionne instantanément le prix de chaque plat pour obtenir le montant total de la commande.

- * Il envoie les détails de la commande en cuisine pour que les cuisiniers puissent commencer la préparation.

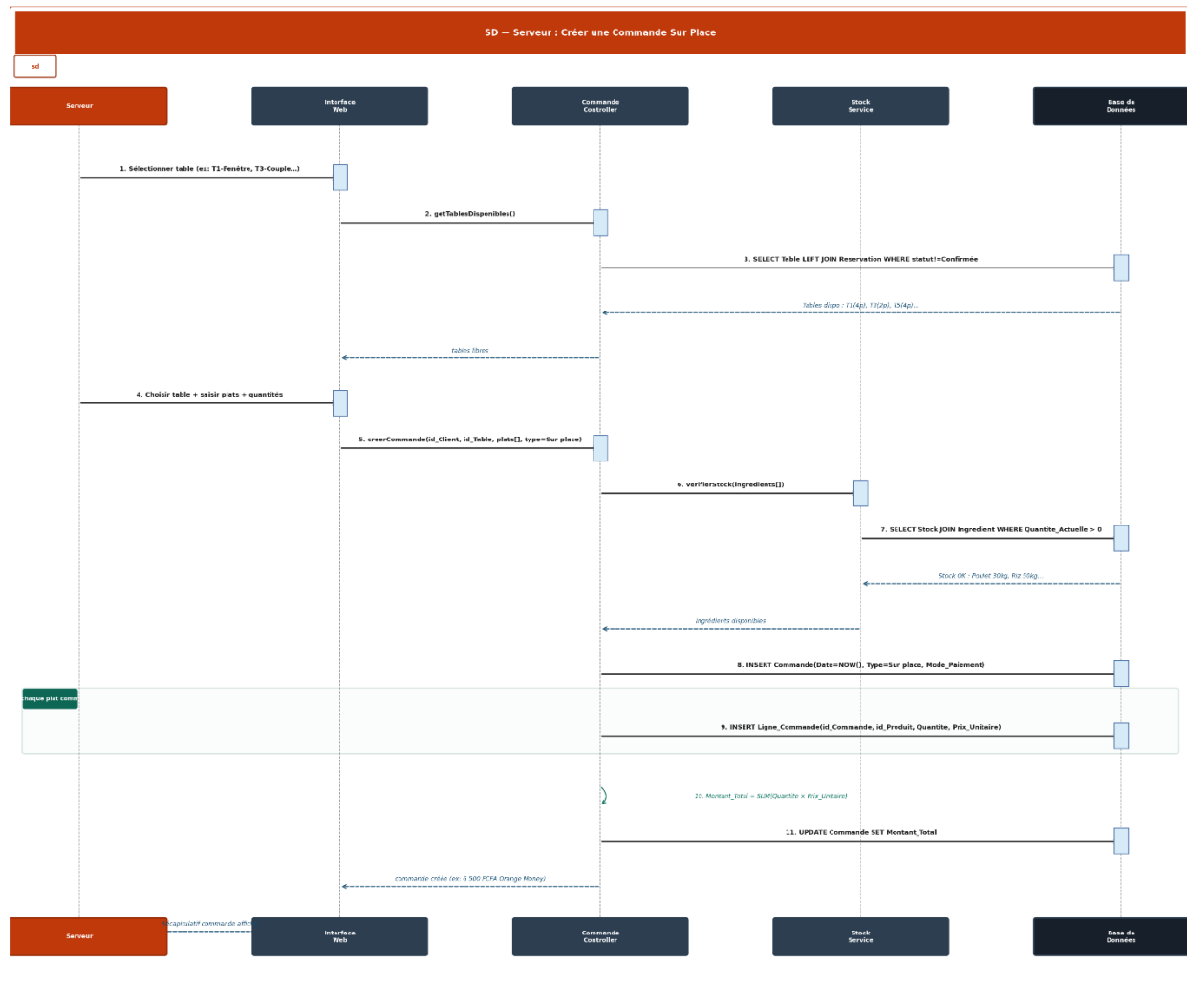


Figure 9: Création d'une commande sur place (D.S serveur)

• Scenario 2 : Générer facture

- * Une fois le repas terminé, le demande au système de produire le document de paiements.
- * Le système rassemble toutes les informations de la commande (plats consommés, boissons, prix unitaires).

- * Il génère une facture officielle associée au client.
- * Il prépare le fichier pour l'impression physique ou l'envoi numérique au client.

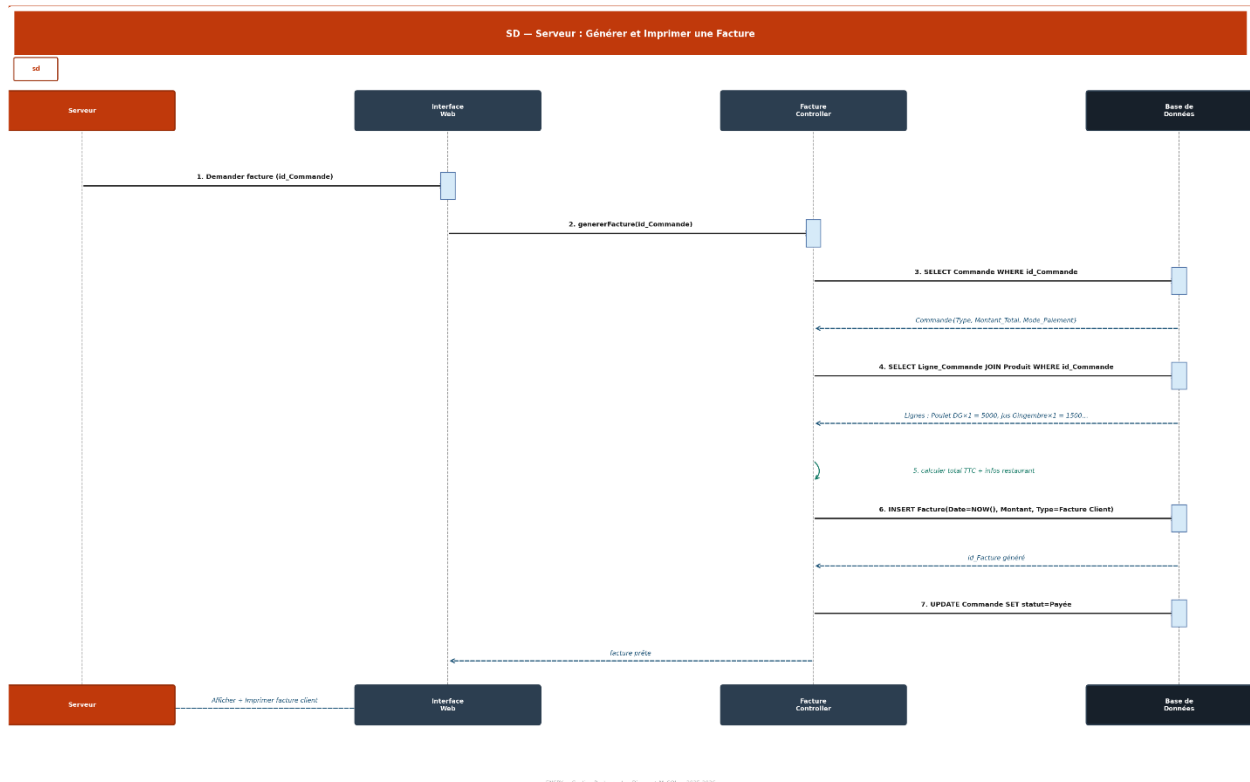


Figure 10: Générer facture (D.S serveur)

• Scenario 3 :_Gérer réservation

- * Le serveur enregistre les demandes des clients qui souhaitent réserver une table à une date et une heure précises
- * Le système consulte le calendrier des tables pour s'assurer que l'emplacement souhaité est disponible à l'heure demandée.
- * Il valide la réservation ou permet son annulation en changeant simplement son statut dans le système.



Figure 11: Gérer réservation(D.S serveur)

- **Scenario 4 : _Gestion de la fiche client**

- * Il crée ou met à jour les informations de contact d'un client (nom, téléphone, email) pour le programme de fidélité ou le suivi.
- * Le système recherche si le client existe déjà pour éviter les doublons.
- * Il mémorise les préférences ou le type de client pour améliorer l'accueil lors des prochaines visites.

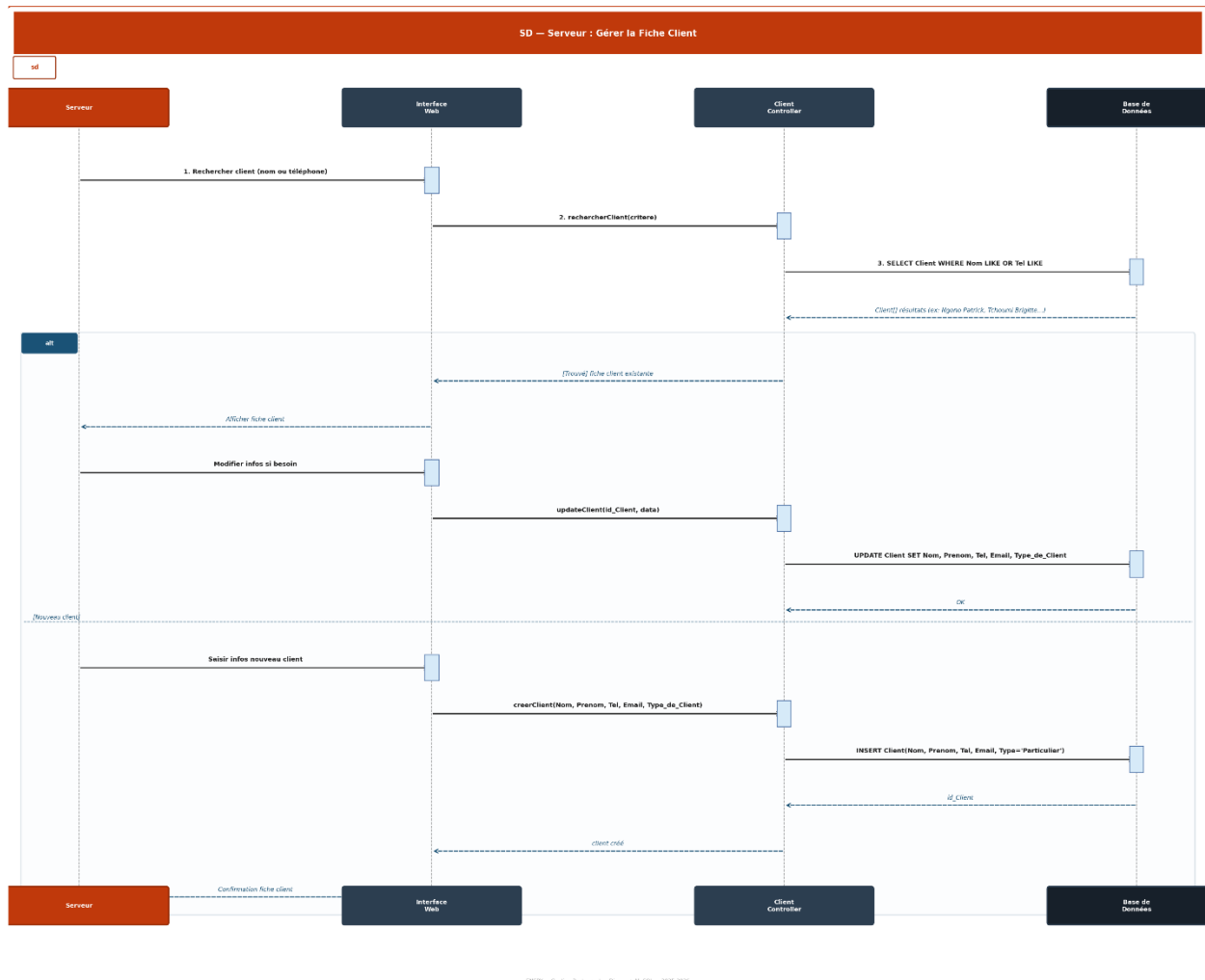


Figure 12: Gestion de la fiche client (D.S serveur)

- ADMINISTRATEUR

L'Administrateur possède les privilèges les plus élevés du système. Son rôle est de garantir la sécurité, de gérer le personnel et de superviser la santé financière globale du restaurant

• Scenario 1 : Authentication sécurisée

- * Il saisit ses identifiants personnels pour accéder à l'interface de gestion.
- * Le système vérifie l'identité dans la base de données et confirme que l'utilisateur possède bien le rang de "Directeur Général

- * Il crée une session sécurisée et enregistre l'heure de connexion dans un journal pour la traçabilité

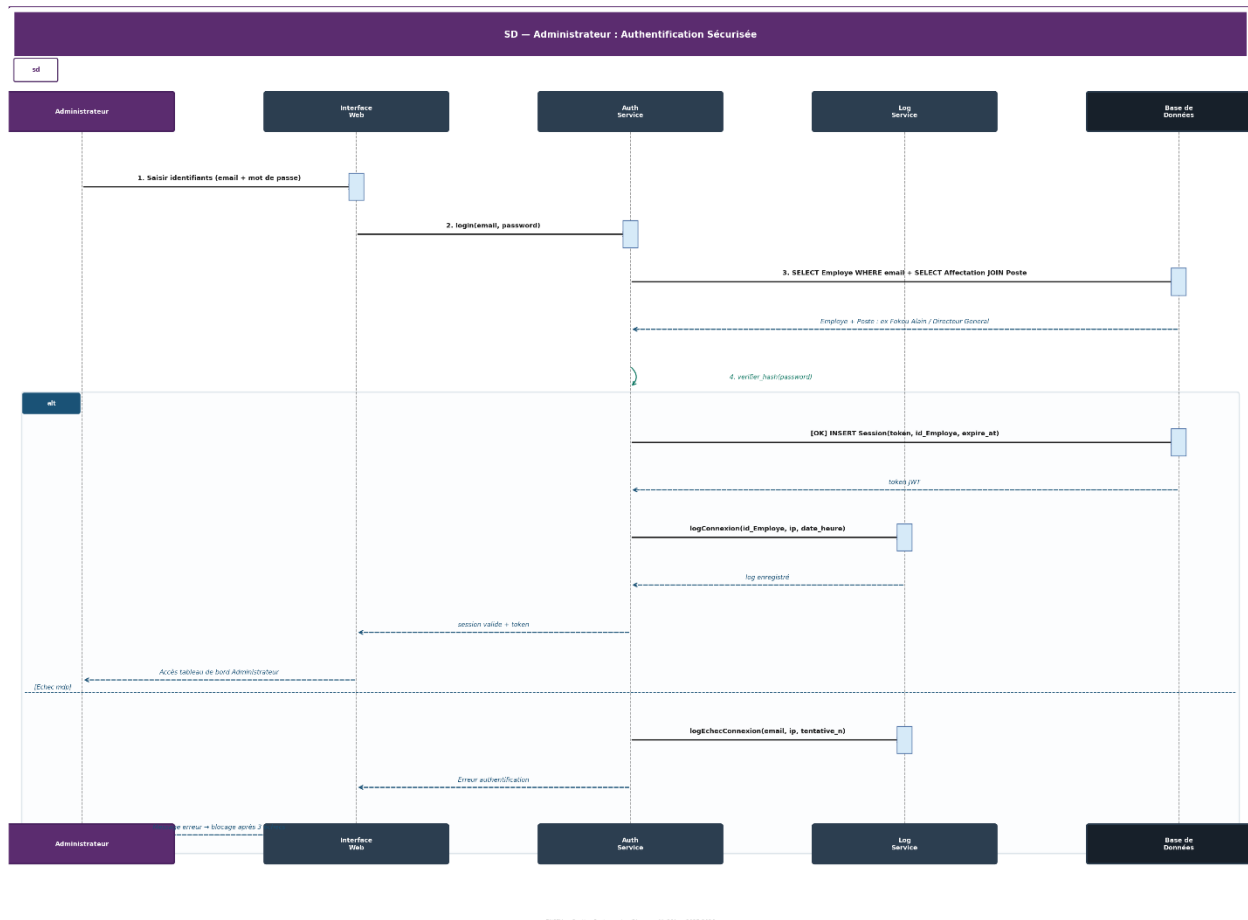


Figure 13: Authentification sécurisée(D.S admin)

- **Scenario 2 : Gestion des utilisateurs et des rôles**

- * Il crée des comptes pour les nouveaux employés et leur attribue un rôle spécifique (parmi les 13 postes disponibles, comme Serveur ou Cuisinier).
- * Le système configure instantanément les permissions de l'employé en fonction de son poste.
- * Il lie l'employé à son affectation pour que chaque action effectuée dans l'application soit signée par son nom.

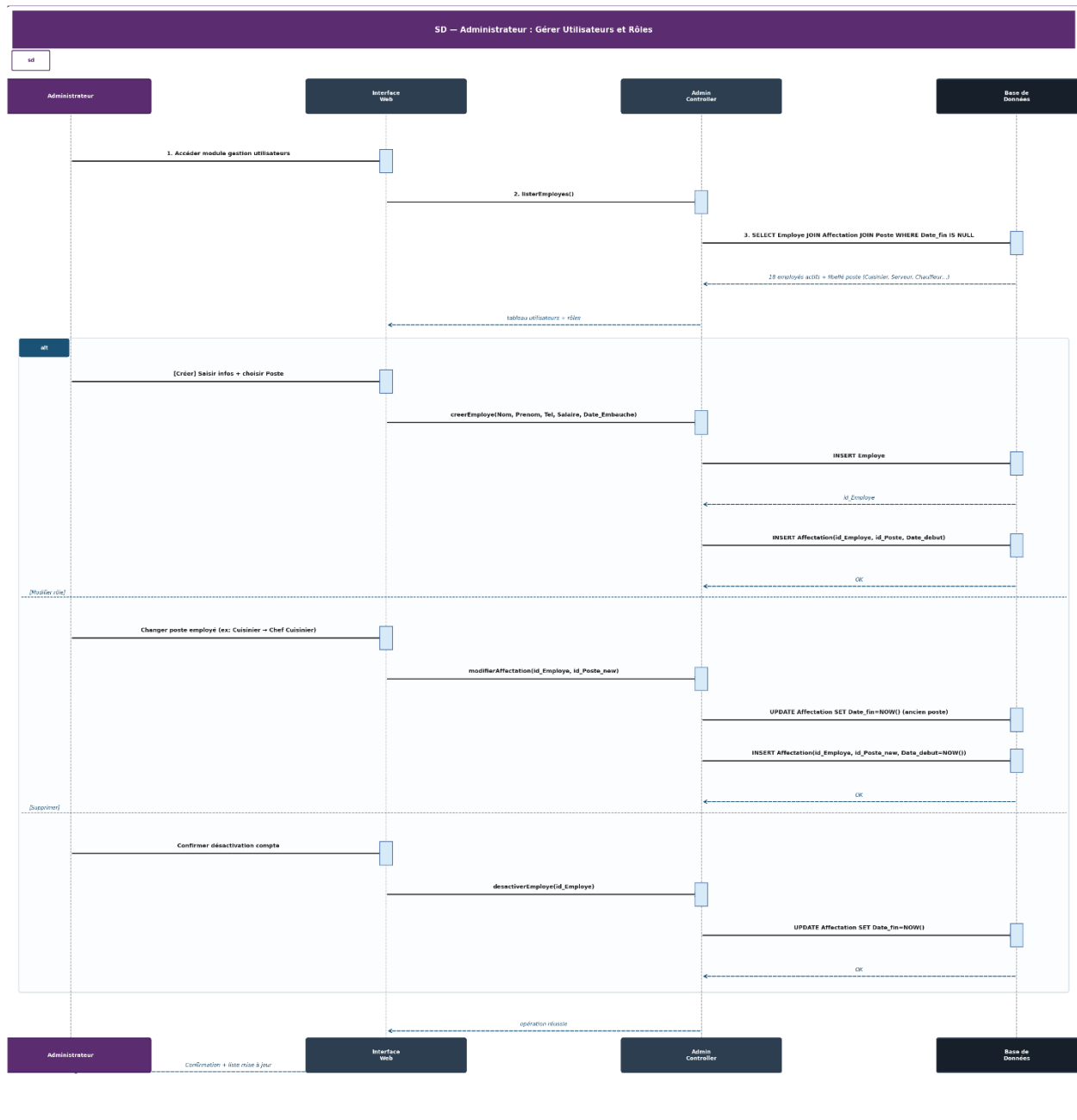


Figure 14: Gestion des utilisateurs et des rôles(D.S admin)

- **Scenario 3 : Gestion des Ressources Humaines (RH) et des salaires**

- * Il consulte les dossiers du personnel et lance le calcul des paies mensuelles.

- * Le système additionne les salaires de tous les employés actifs pour obtenir le montant total à verser.
- * Il génère une facture de type "Paieement Salaires" dans la gestion financière pour que cette dépense soit visible dans les rapports

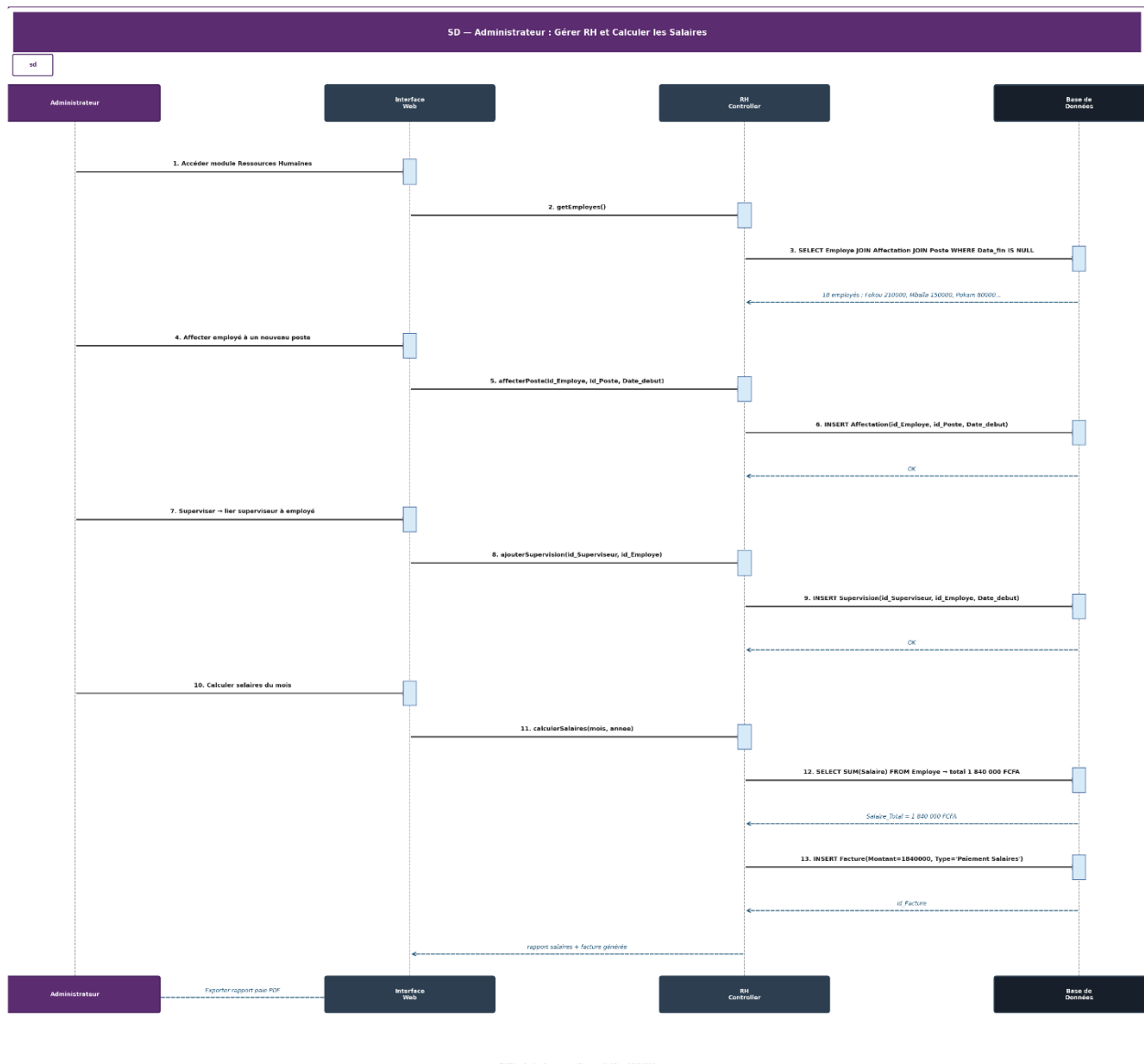


Figure 15: Gestion des Ressources Humaines (RH) et des salaires (D.S admin)

- **Scenario 4 : Configuration globale du système**

- * Il modifie les paramètres généraux, comme la liste des équipements ou les types de postes disponibles.
- * Le système applique immédiatement les nouveaux paramètres à l'ensemble de l'application (ex : un nouvel équipement devient disponible pour le gestionnaire de stock).
- * Il mémorise ces changements de structure dans la base de données MySQL.

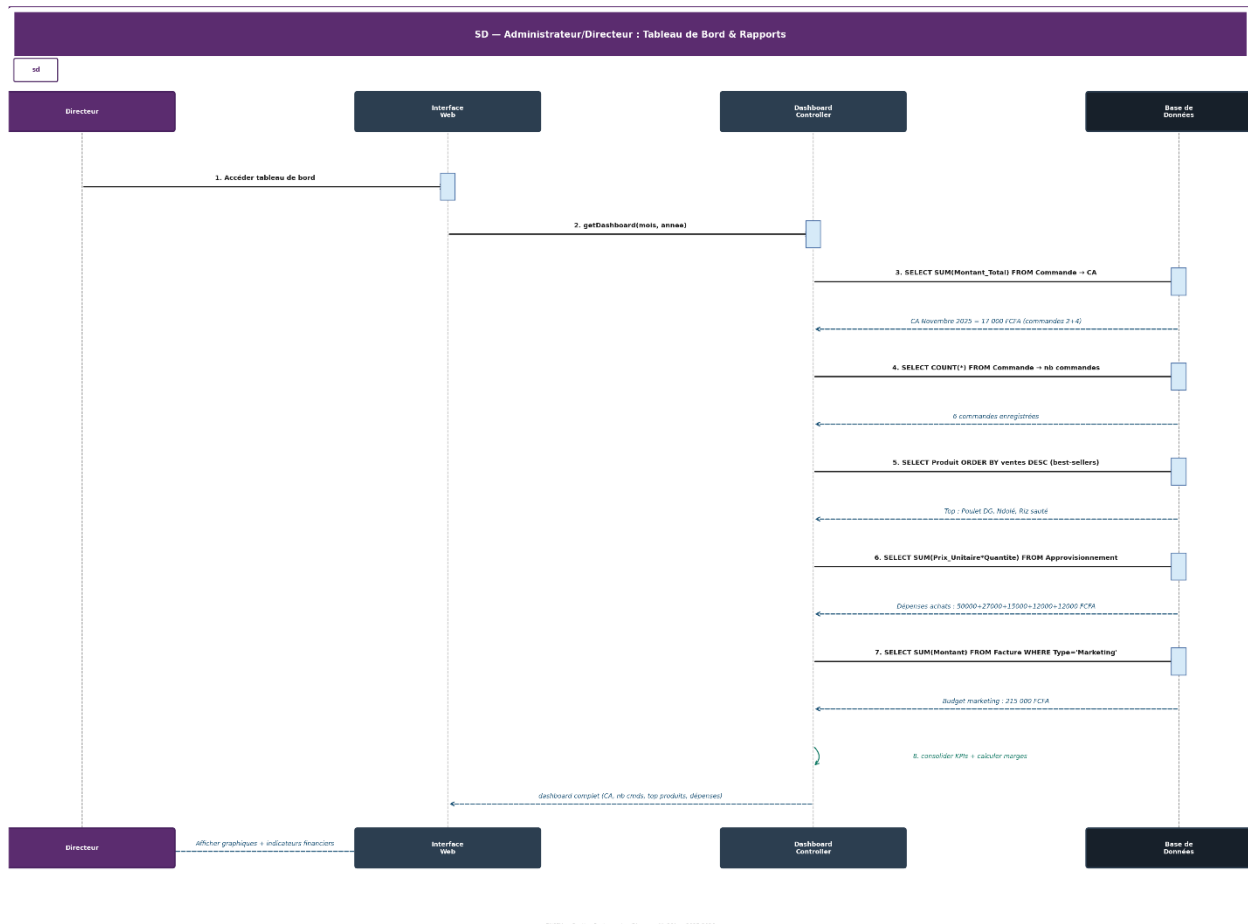


Figure 16: Configuration globale du système (D.S admin)

- CHEF CUISINIER

- **Scenario 1 : création d'une recette**

- * Le chef cuisinier remplit le formulaire avec le nom du plat, les ingrédients et les quantités.
- * Le système vérifie que tous les champs obligatoires sont remplis et que les quantités sont cohérentes (ex : pas de poids négatif).
- * Le système crée une nouvelle entrée dans la base de données MySQL pour le produit et lie les ingrédients correspondants.
- * Le système affiche un message de succès ("Recette créée avec succès") et redirige le Chef vers la liste des menus mis à jour.

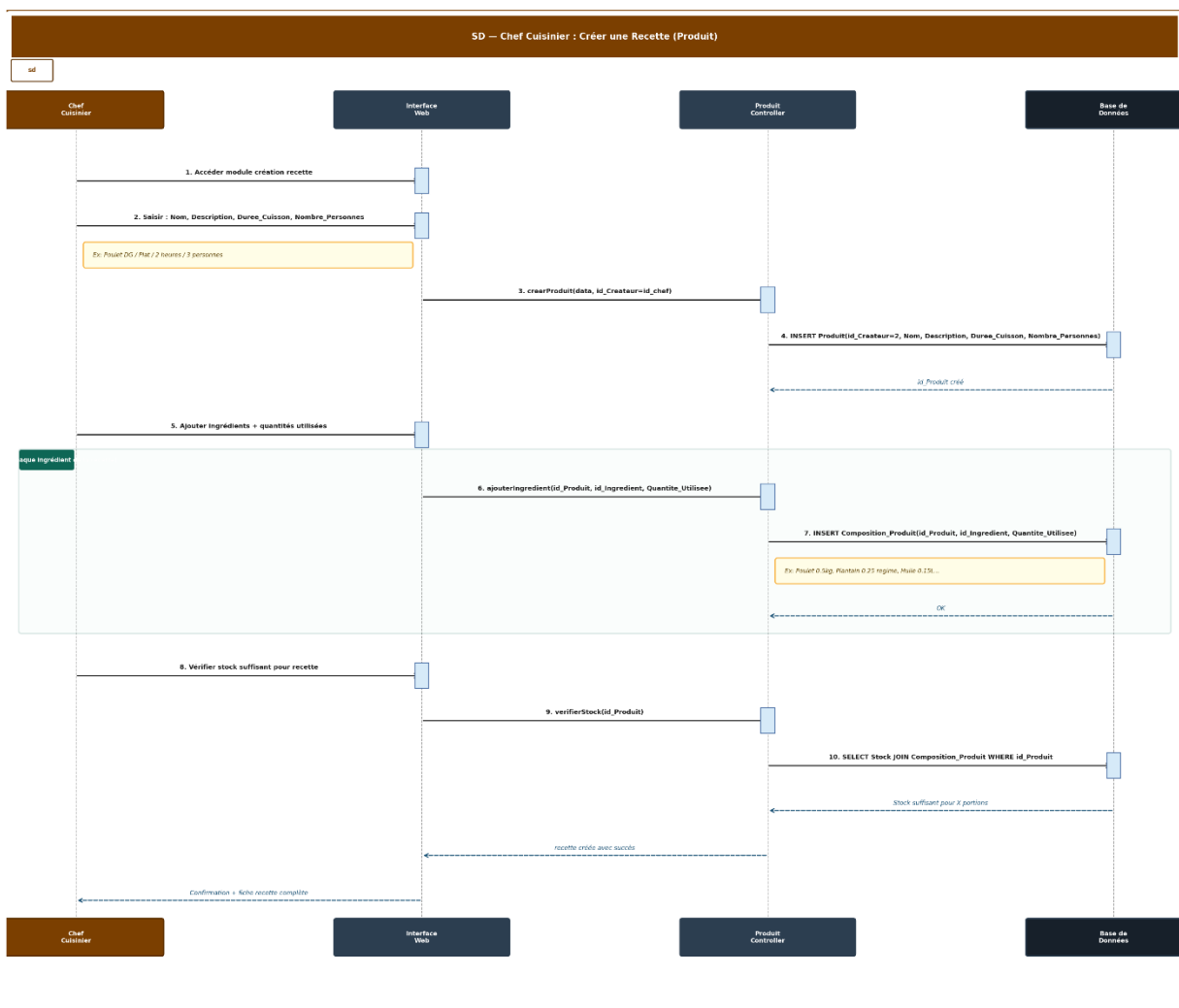


Figure 17: création d'une recette (D.S chef)

- **Scenario 2 : modification d'une recette**

- * Le chef modifie un élément (ex : change la durée de cuisson ou retire un ingrédient).
- * Le système remplace les anciennes informations par les nouvelles dans la base de données.
- * Si un ingrédient est modifié, le système peut recalculer automatiquement le coût de revient du plat pour aider à l'optimisation des coûts.
- * Le système confirme la modification et met à jour instantanément l'affichage pour les autres utilisateurs (comme le Serveur qui consulte le menu).

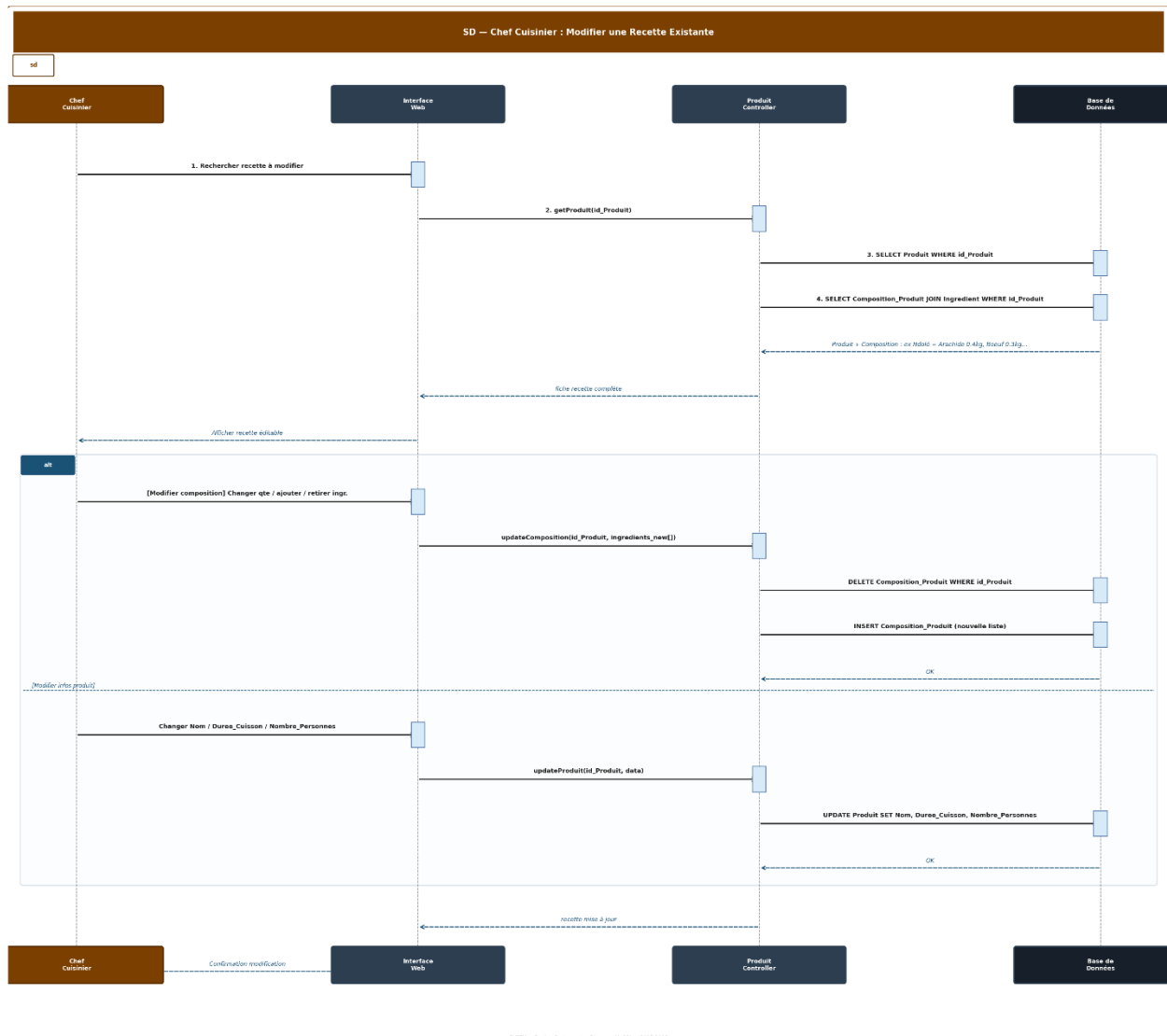


Figure 18: modification d'une recette (D.S chef)

• **Scenario 3 : supervision de la cuisine**

- * Le chef consulte les commandes et assigne un cuisinier à une tâche
- * Il extrait de la base de données uniquement les commandes dont l'état est "En attente" ou "En préparation".
- * Il met à jour le statut de la commande en y associant l'identifiant du cuisinier choisi pour assurer la traçabilité.

- * Il envoie une notification sur l'interface du Cuisinier concerné pour l'informer qu'une nouvelle tâche lui a été attribuée.

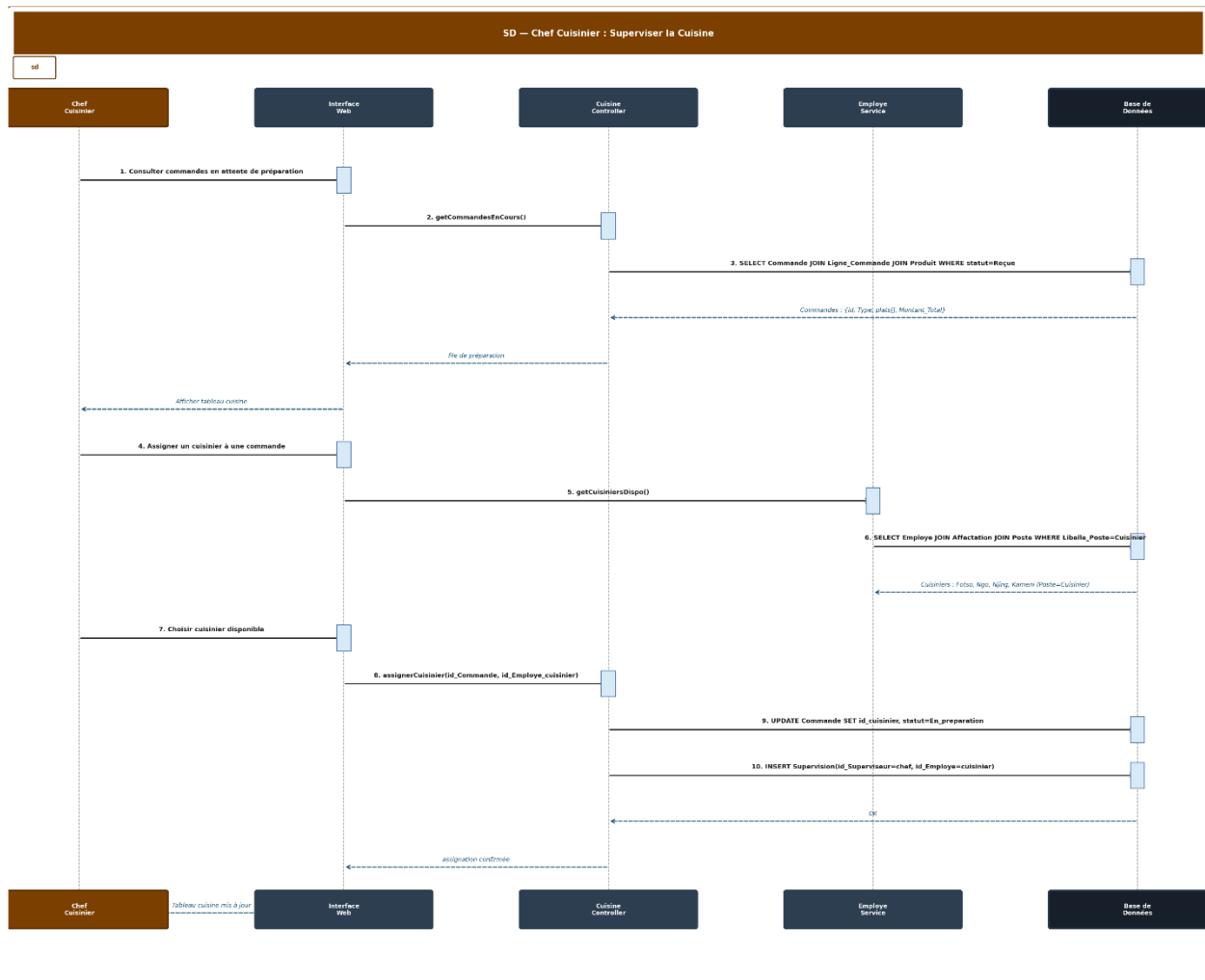


Figure 19: supervision de la cuisine (D.S chef)

- CUISINIER

• Scenario 1 : Préparation et validation d'une commande

- * Il consulte la liste des commandes à préparer, cuisine les plats demandés, puis marque la commande comme "Prête" dans l'application.
- * Le système rassemble et affiche tous les ingrédients et les quantités nécessaires pour la recette précise.

- * Dès que le plat est validé, le système déduit automatiquement les quantités d'ingrédients utilisées du stock global.
- * Il crée une trace de cette "sortie de stock" dans l'historique des variations pour permettre un suivi précis des consommations.

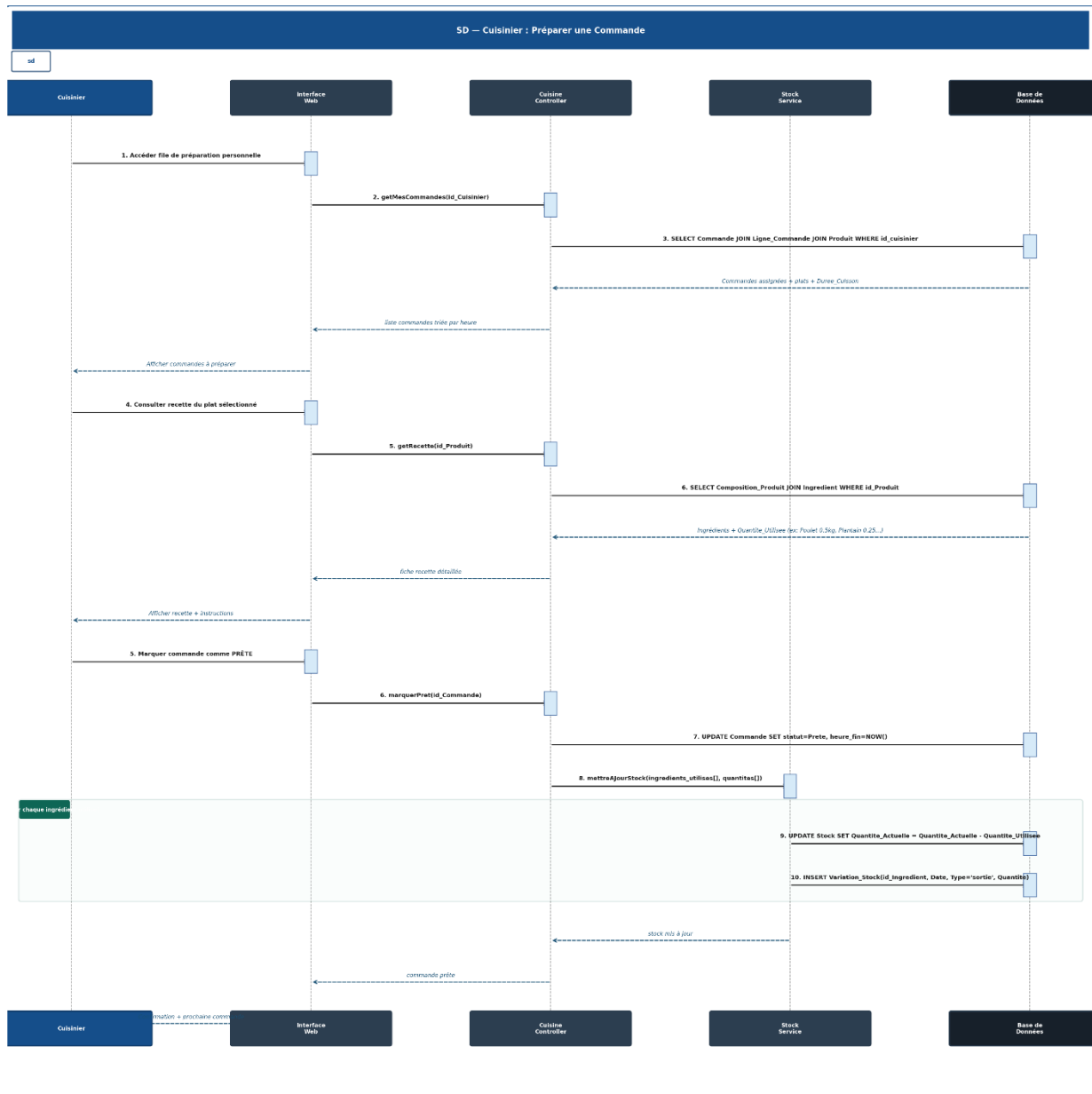


Figure 20: Préparation et validation d'une commande(D.S cuis)

• Scenario 2 : Signalement d'une rupture de stock

- * S'il constate qu'un ingrédient est épuisé physiquement en cuisine, il le signale immédiatement via son interface.
- * Le système force la quantité de cet ingrédient à zéro dans la base de données pour éviter que d'autres commandes de ce plat ne soient prises.
- * L'envoi d'une notification instantanée au **Gestionnaire de stock** pour demander un réapprovisionnement urgent.
- * Le système enregistre l'événement comme une rupture critique dans l'historique des variations.

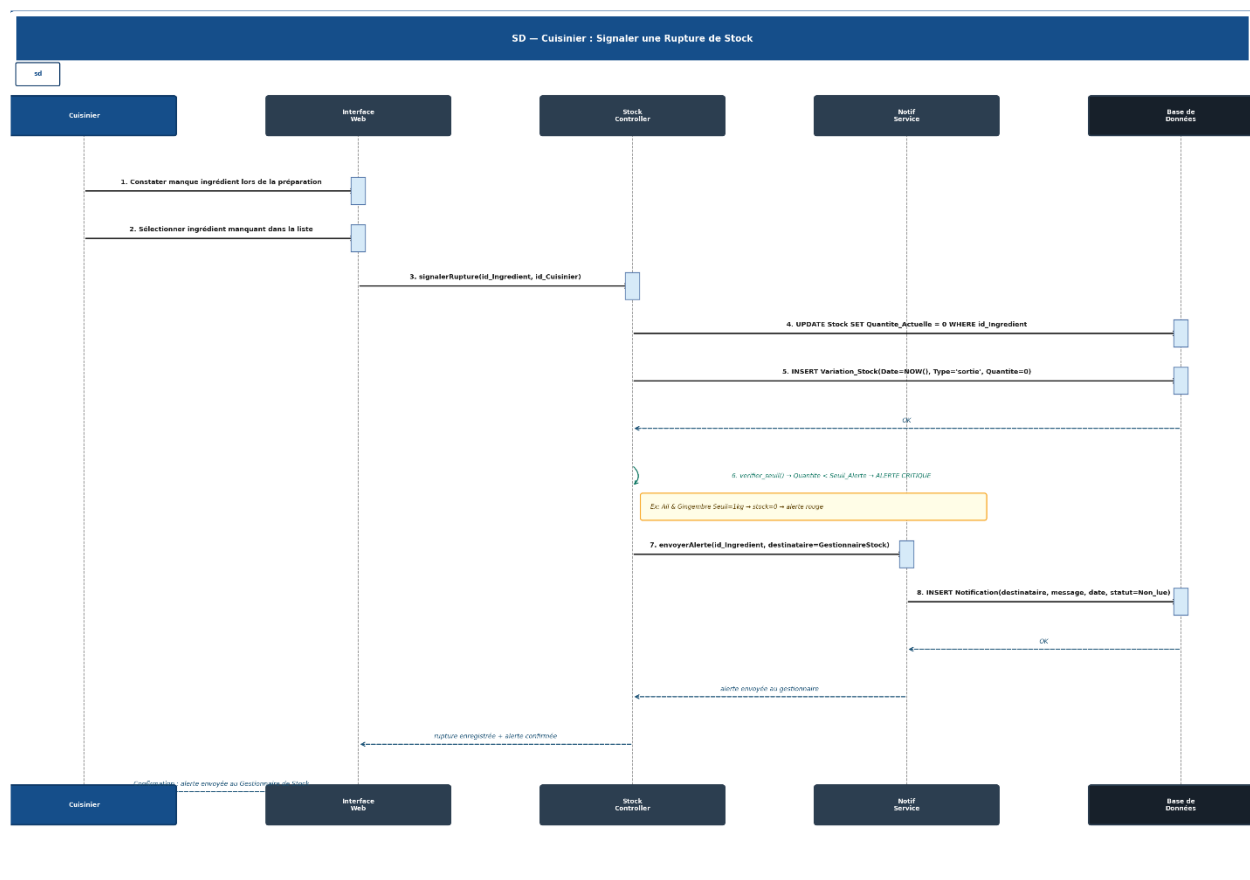


Figure 21: Signalement d'une rupture de stock (D.S cuis)

- **GESTIONNAIRE DE STOCKS**

- **Scenario 1 : Consultation et mise à jour du stock**

- * Il consulte l'état des réserves et ajuste manuellement les quantités si nécessaire (par exemple, après un inventaire physique).
- * Le système affiche en priorité les ingrédients dont la quantité est inférieure ou égale au seuil d'alerte défini.
- * Il met à jour les quantités en base de données dès que la modification est validée.
- * Il enregistre chaque ajustement dans l'historique des variations pour assurer la traçabilité des mouvements de stock.

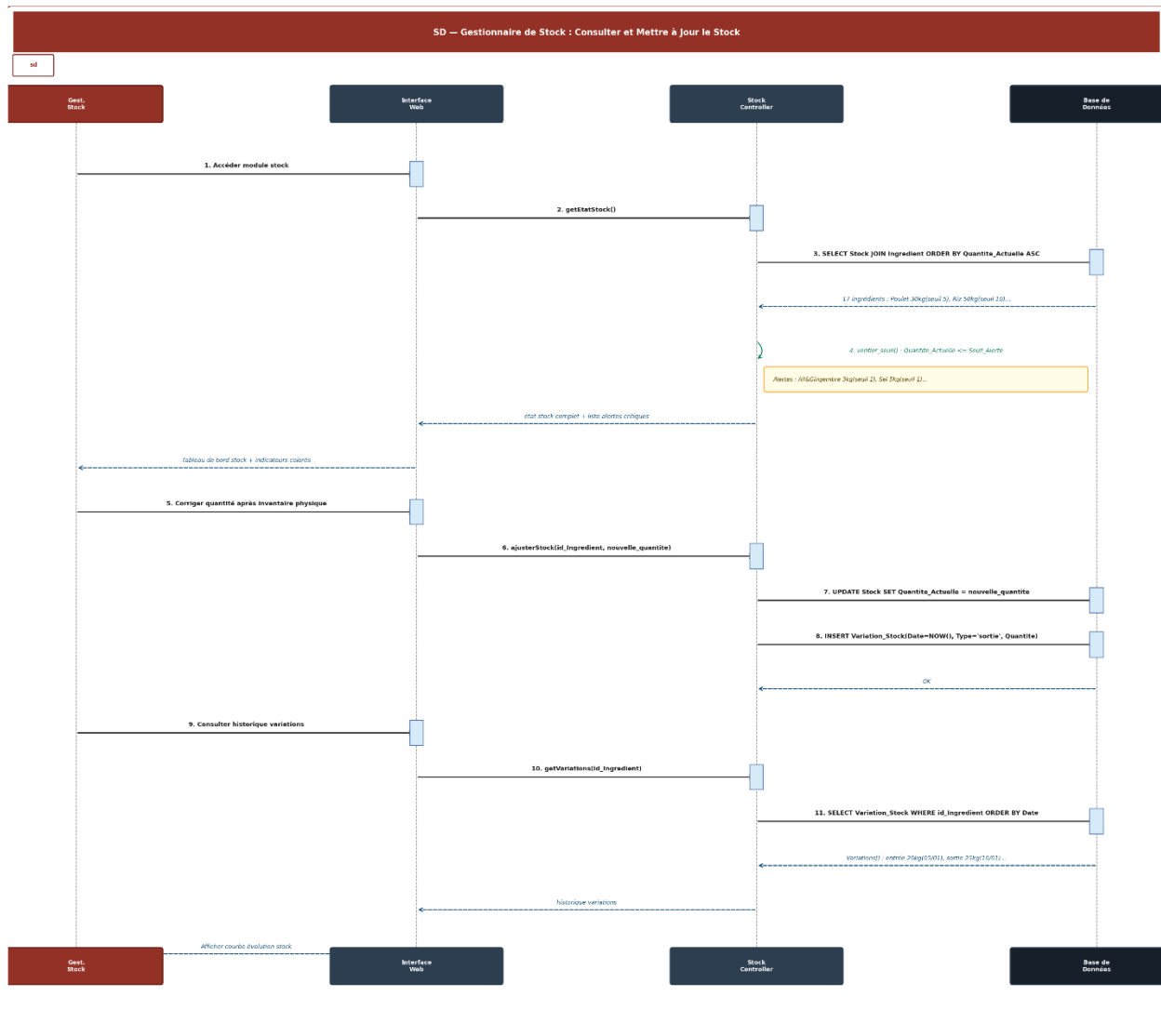


Figure 22: Consultation et mise à jour du stock (D.S gest)

• **Scenario 2 : Passage de commande aux fournisseurs**

- * Le gestionnaire sélectionne un fournisseur et enregistre les nouveaux achats d'ingrédients en précisant les quantités et les prix.
- * Le système crée un nouvel événement de déplacement incluant le chauffeur et le véhicule pour aller chercher les marchandises.
- * Il génère automatiquement une facture d'achat pour documenter la dépense d'approvisionnement dans la comptabilité du restaurant.

- * Il prépare l'entrée des marchandises dans le système pour qu'elles soient ajoutées aux stocks dès leur réception.

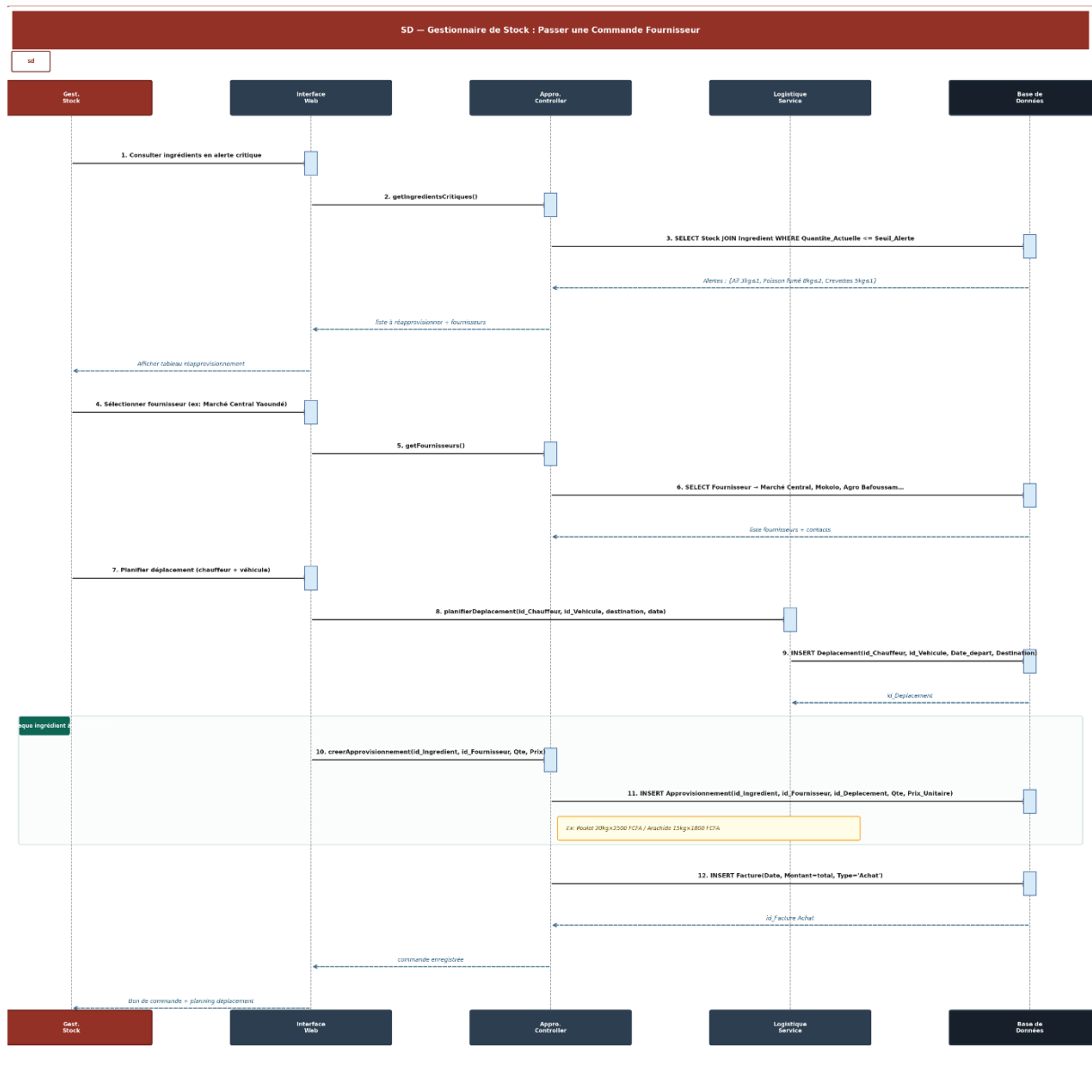
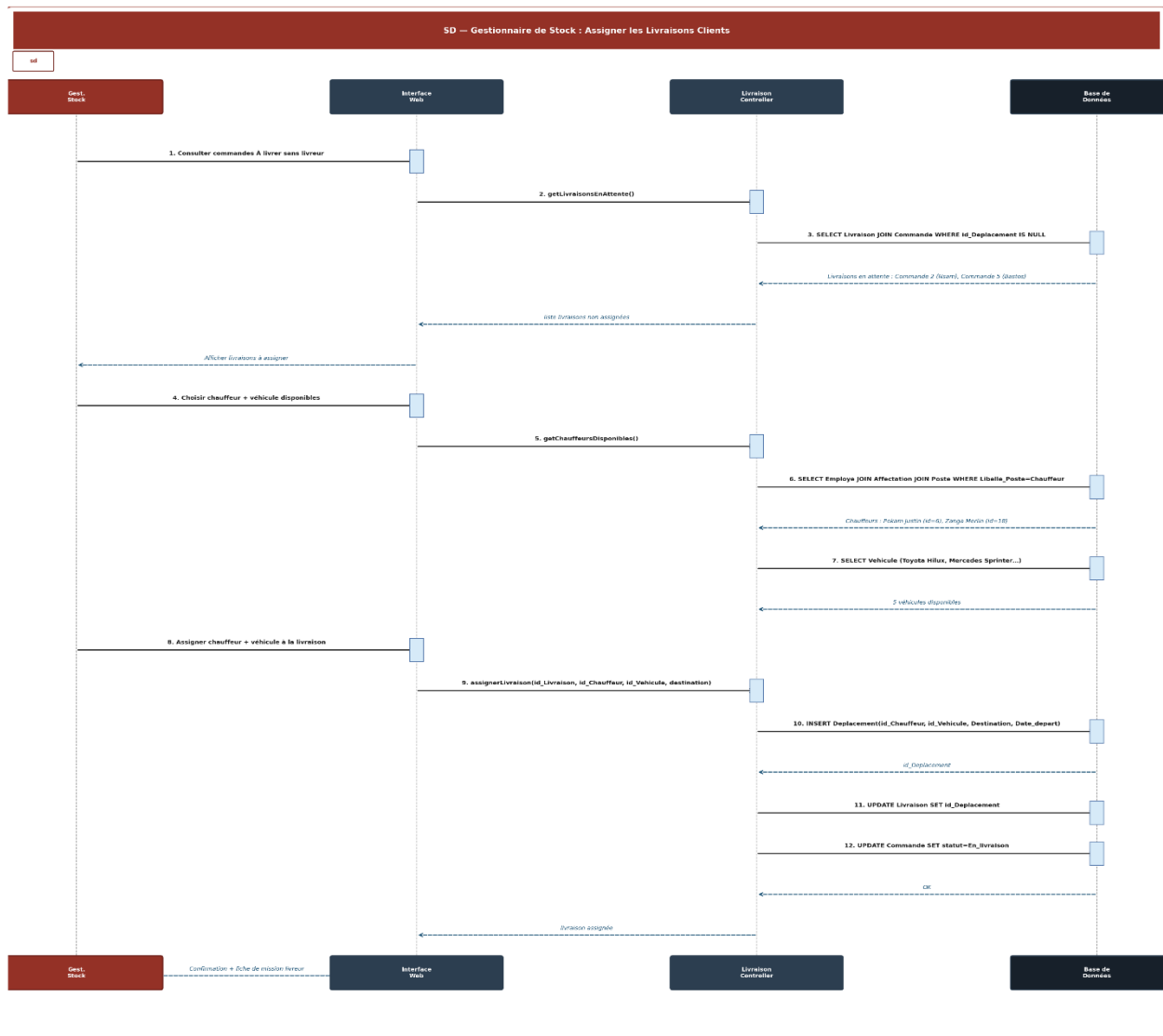


Figure 23: Passage de commande aux fournisseurs (D.S gest)

- **Scenario 3 : Assignment des livraisons clients**

- * Le gestionnaire organise les livraisons à domicile en attribuant un véhicule et un itinéraire aux commandes prêtes.
- * Le système liste toutes les livraisons qui n'ont pas encore de transport assigné.
- * Il lie la commande au déplacement créé et informe le **Livreur** qu'une nouvelle course est prête à être effectuée.



- **LE LIVREUR**

Le Livreur est le dernier maillon de la chaîne de service. Son rôle est de s'assurer que les commandes arrivent à destination dans les meilleures conditions et de confirmer la clôture du processus de vente.

- **Scenario 1 : Prise en charge d'une livraison**

- * Il consulte sur son interface les courses qui lui ont été attribuées et récupère les informations nécessaires au trajet
- * Le système filtre les déplacements pour n'afficher que ceux liés à l'identifiant du livreur connecté.
- * Il extrait et affiche l'adresse de destination ainsi que les contacts du client pour faciliter la livraison.
- * Le système marque la commande comme "En cours de livraison" pour informer le restaurant et le client.

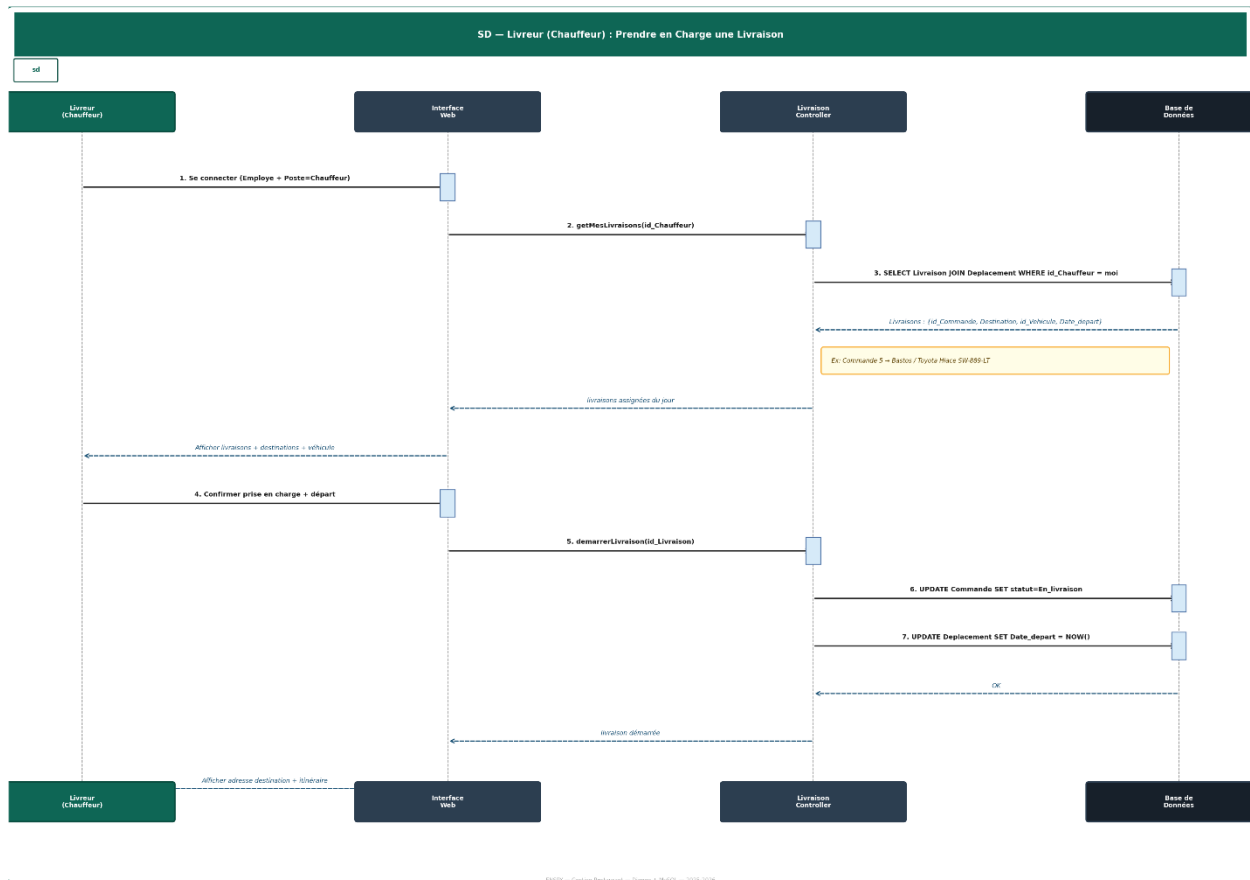


Figure 25: Prise en charge d'une livraison

• **Scenario 2 : Confirmation de la livraison**

- * Une fois le colis remis au client, il valide la fin de la course sur son application.
- * Le système met à jour le statut de la commande en "Livré" dans la base de données.
- * Il enregistre la distance totale parcourue dans le journal des déplacements pour le suivi logistique.
- * Il envoie automatiquement un message de confirmation au client pour le remercier et clore la transaction.

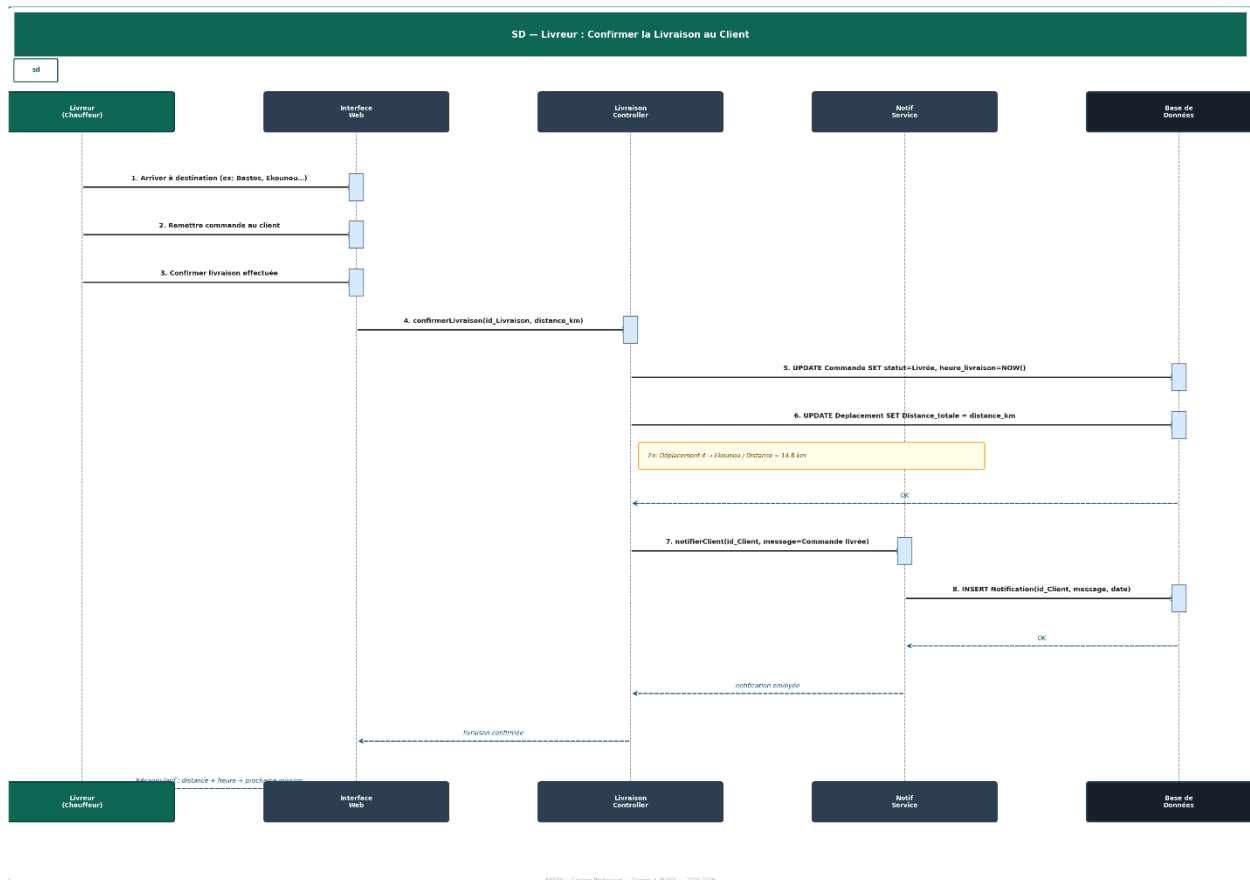


Figure 26: Confirmation de la livraison(D.S livr)

2.2.4. DIAGRAMME DE DÉPLOIEMENT

Le diagramme de déploiement est un diagramme structurel d'UML qui décrit l'architecture physique d'un système en représentant la distribution logicielle sur les ressources matérielles. Il modélise les nœuds (serveurs, ordinateurs, capteurs, ou périphériques), les artefacts (fichiers exécutables, bibliothèques, bases de données) qui y sont installés, ainsi que les protocoles de communication connectant ces composants. Ses objectifs principaux sont de planifier la topologie du réseau, de spécifier la configuration matérielle nécessaire au fonctionnement de l'application et de résoudre les problématiques de performance, de scalabilité et de sécurité liées à la mise en production réelle du système.

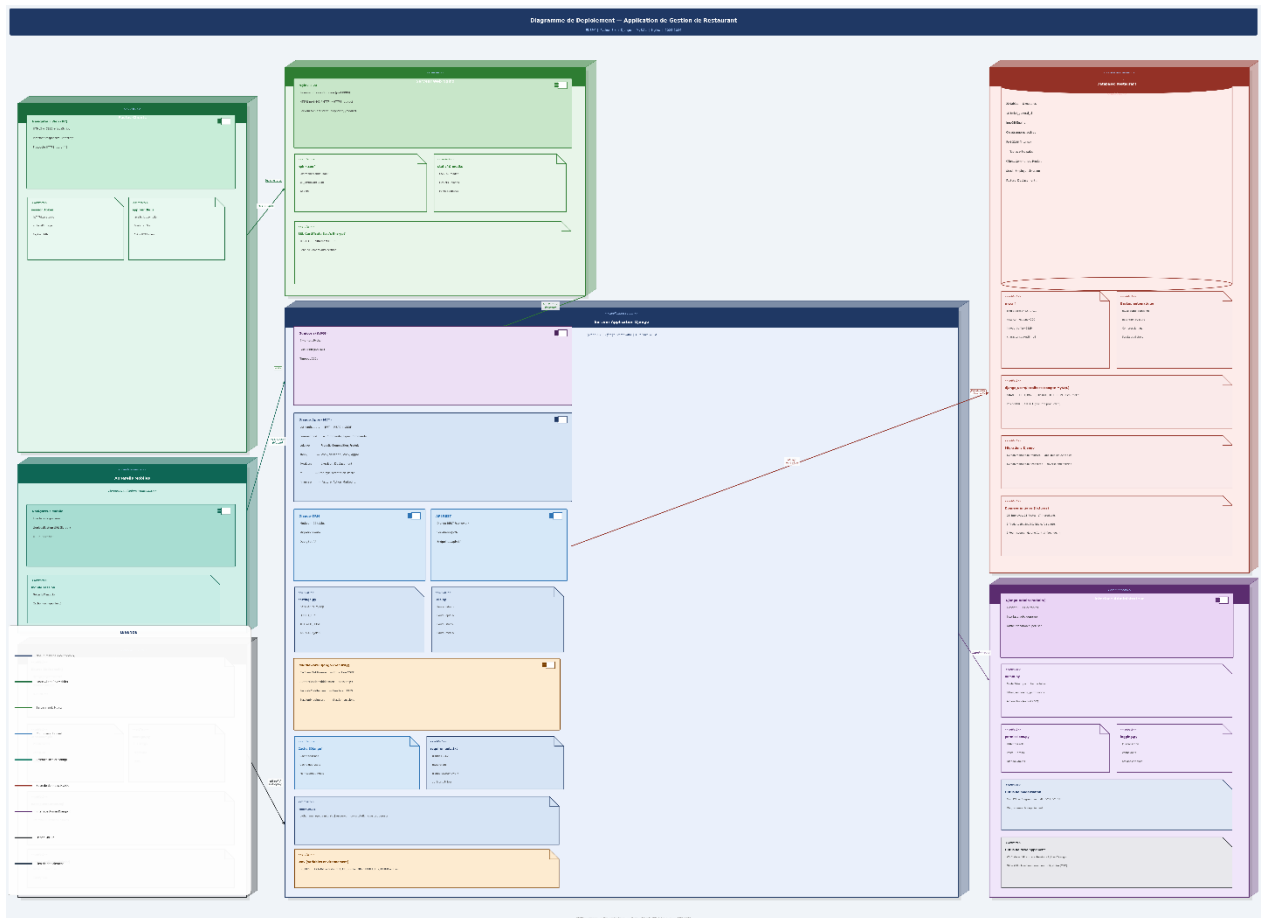


Figure 27: Diagramme de déploiement

L'architecture du système repose sur une structure décentralisée où les composants logiciels sont répartis sur différents nœuds matériels pour garantir sécurité, performance et scalabilité.

- **Poste Client (Navigateur Web)** Les 7 acteurs accèdent au système via un navigateur (Chrome, Firefox). Le client utilise HTML/CSS/JS avec des requêtes HTTP/HTTPS vers le serveur Django.
- **Serveur d'Application Django** Cœur du système. Il héberge le Framework Django (Python 3.x) avec l'architecture MVT. Il contient les apps : authentication, commandes, cuisine, stock, livraison, finances, RH. Il expose une API REST et gère les sessions via tokens JWT. Il communique avec la base via Django ORM.

- **Serveur de Base de Données MySQL** Héberge la base Restaurant. Contient les 22 tables réparties en 8 modules. Accessible uniquement depuis le serveur Django (port 3306, connexion sécurisée).
- **Serveur Web Nginx** Reverse proxy devant Django. Sert les fichiers statiques (CSS, JS, images). Gère le SSL/TLS (HTTPS port 443). Redirige les requêtes dynamiques vers Gunicorn.
- **Appareils mobiles** : Les livreurs et cuisiniers accèdent via smartphone (interface responsive Bootstrap).
- **GitHub** Hébergement du code source, versionnement, collaboration équipe (obligatoire selon cahier des charges).

2.2.5. DIFFICULTES RENCONTREES

Nous avons eu des difficultés dans l'installation des outils de conception et dans la réalisation des différents diagrammes. Nous avons pu résoudre ses difficultés en visionnant des vidéos YouTube

2.3. TRAVAIL EFFECTUE PAR CHAQUE MEMBRE DU GROUPE

Tableau 6: Répartition des taches

Membres du groupe	Travail effectué
Etolo Philippe	Diagramme de cas d'utilisation et de classe, Diagramme de séquence(serveur, gestionnaire de stock), Diagramme d'activités, méthodes de développements
Djouma Noé	Diagramme de séquence (client, Administrateur, serveur), Diagramme d'activité, méthodes de développements
Meko'o Emvolo	Diagramme de séquence(Chef cuisinier, cuisinier, serveur, gestionnaire de stock), diagramme d'activité , méthodes de développements
Bomo Akam	Diagramme de séquence (livreur)
YOUWE LAMNA SYLVAIN	/

PARTIE III : IMPLÉMENTATION DES MODULES DE L'APPLICATION

Cette partie présente les réalisations techniques effectuées pour chacun des modules définis dans le cahier des charges. Pour chaque module, nous décrivons les fonctionnalités implémentées, les difficultés rencontrées ainsi que les solutions adoptées.

3.1 MODULE DE GESTION DES UTILISATEURS ET DES RÔLES

3.1.1 Fonctionnalités réalisées

Ce module constitue le socle sécuritaire de l'application. Il gère les comptes utilisateurs et la hiérarchie des droits d'accès.

- Authentification par email et mot de passe.
- Gestion de six (6) profils utilisateurs : Administrateur, Directeur, Chef Cuisinier, Cuisinier, Serveur et Gestionnaire de Stock.
- Attribution de permissions par rôle via les groupes Django (`django.contrib.auth.models.Group`).
- Interface d'administration permettant la création, modification, désactivation et suppression des comptes employés.
- Redirection automatique vers le tableau de bord approprié selon le rôle après connexion.
- Protection CSRF sur tous les formulaires de connexion.
- Journal des connexions : enregistrement de l'heure et de l'IP pour la traçabilité.
- Gestion des sessions avec expiration automatique après 30 minutes d'inactivité.

3.1.2 Difficultés rencontrées et solutions

Tableau 7: Module de gestion des utilisateurs

Difficulté rencontrée	Solution adoptée
Le modèle User de Django n'utilise pas l'email comme identifiant par défaut mais le username.	Création d'un backend personnalisé (EmailAuthBackend) héritant de ModelBackend.
Attribution de permissions granulaires par rôle sans répétition de code.	Utilisation des Groupes Django : 6 groupes (un par rôle) avec permissions associées.
Redirection après login vers des dashboards différents selon le rôle.	Surcharge de LOGIN_REDIRECT_URL via une vue qui inspecte les groupes de l'utilisateur connecté.

3.2 MODULE DE GESTION DES RECETTES

3.2.1 Fonctionnalités réalisées

- Création d'une recette (Produit) avec : nom, description, durée de cuisson, nombre de personnes, prix de vente.
- Association ingrédient-recette via la table Composition_Produit avec quantité nécessaire.
- Modification d'une recette existante : ajout/suppression d'ingrédients, mise à jour des quantités.
- Suppression logique d'une recette (marquage comme inactive plutôt que suppression physique).
- Vérification automatique de la cohérence des quantités (pas de valeurs négatives ou nulles).
- Désignation du chef responsable de chaque recette.

3.2.2 Difficultés rencontrées et solutions

Tableau 8 :difficultés module de gestion des recettes

Difficulté rencontrée	Solution adoptée
La relation Many-to-Many entre Produit et Ingredient avec un attribut supplémentaire (quantité) n'est pas directement supportée par ManyToManyField.	Création d'un modèle intermédiaire explicite Composition_Produit avec les champs id_produit, id_ingredient et quantite_utilisee.
Les formulaires dynamiques (plusieurs ingrédients en une soumission) complexifient la validation côté Django.	Utilisation des Formsets Django (formset_factory) pour gérer plusieurs lignes d'ingrédients avec validation individuelle.
Risque de suppression d'une recette encore utilisée dans des commandes en cours.	Vérification avant suppression : si la recette est référencée dans des Lignes_Commandes actives, la suppression est bloquée.

3.3 MODULE DE GESTION DES PRODUITS ET DU CATALOGUE

3.3.1 Fonctionnalités réalisées

- Catalogue des plats avec catégorisation (Entrées, Plats principaux, Desserts, Boissons).
- Affichage du menu avec photos et descriptions pour les clients.
- Tarification dynamique : modification du prix d'un plat sans toucher à la recette associée.
- Filtrage et recherche dans le catalogue par catégorie, nom ou fourchette de prix.

3.3.2 Difficultés rencontrées et solutions

Tableau 9: difficultés module de gestion des produits et du catalogue

Difficulté rencontrée	Solution adoptée
L'indicateur de disponibilité nécessite une vérification des stocks à chaque affichage, ce qui ralentit la page.	Ajout d'un champ booléen <code>est_disponible</code> sur le modèle <code>Produit</code> , mis à jour par un signal Django à chaque modification de stock.
Les images uploadées occupaient trop d'espace disque.	Intégration de Pillow pour compresser les images dépassant 500 Ko et les convertir en JPEG optimisé à l'enregistrement.

3.4 MODULE DE GESTION DES COMMANDES

3.4.1 Fonctionnalités réalisées

- Création de commandes sur place : le serveur sélectionne la table et les plats depuis l'interface numérique.
- Création de commandes en ligne : le client passe sa commande depuis son espace personnel avec livraison à domicile.
- Calcul automatique du montant total à partir des lignes de commande et des prix unitaires.
- Suivi du statut en temps réel : En attente → En préparation → Prête → Livrée / Payée.
- Génération de factures PDF téléchargeables.
- Annulation de commande avec justification et traçabilité.

3.4.2 Difficultés rencontrées et solutions

Tableau 10: Difficultés module de gestion des produits et du catalogue

Difficulté rencontrée	Solution adoptée
La validation implique plusieurs opérations simultanées (création commande, insertion lignes, décrémentation stock). Une panne laisserait la base incohérente.	Utilisation de <code>django.db.transaction.atomic()</code> pour encapsuler toutes les opérations. En cas d'erreur, un rollback automatique annule tout.
Le suivi du statut en temps réel nécessitait des rechargements fréquents de page.	Mécanisme de polling léger en JavaScript (<code>setInterval</code> toutes les 30 secondes) pour rafraîchir uniquement le badge de statut.

3.5 MODULE DE GESTION DES STOCKS

3.5.1 Fonctionnalités non réalisées

- Consultation en temps réel du stock de chaque ingrédient (quantité actuelle, unité, seuil d'alerte).
- Mise à jour automatique du stock après chaque commande validée et préparée en cuisine.
- Alertes visuelles dans le tableau de bord du gestionnaire quand un ingrédient passe sous son seuil critique.
- Signalement de rupture de stock par le cuisinier avec notification au gestionnaire.
- Historique complet des variations de stock (entrées/sorties) avec horodatage et motif.
- Gestion des fournisseurs : création, modification, liste des contacts.
- Passation de commandes fournisseurs avec génération de facture d'achat.

3.6 MODULE DE GESTION DES RESSOURCES HUMAINES (RH)

3.6.1 Fonctionnalités réalisées

- Gestion des fiches employés : nom, prénom, contact, date d'embauche, salaire de base.
- Attribution et modification des postes parmi les 13 postes disponibles dans le système.
- Suivi des affectations avec dates de début et de fin pour chaque poste.
- Calcul automatique de la masse salariale mensuelle avec rapport de paie.
- Désactivation de compte employé avec conservation de l'historique.
- Tableau de bord RH : effectif actif, répartition par poste, charges salariales.

3.6.2 Difficultés rencontrées et solutions

Tableau 11: Difficultés module de gestion des ressources humaines (rh)

Difficulté rencontrée	Solution adoptée
La relation de supervision (un employé supervise plusieurs autres) créait une relation récursive sur la même table.	Modélisation avec une table Supervision séparée (id_superviseur, id_employe, date_debut) permettant l'historisation.
La génération des bulletins de paie pour tous les employés pouvait dépasser le timeout HTTP.	Utilisation d'une vue avec StreamingHttpResponse pour envoyer progressivement les PDF générés.

3.7 MODULE TABLEAU DE BORD ET RAPPORTS FINANCIERS

3.7.1 Fonctionnalités réalisées

- Tableau de bord principal (Administrateur/Directeur) avec indicateurs clés de performance (KPI).
- Chiffre d'affaires mensuel calculé dynamiquement à partir des commandes payées.

- Classement des produits les plus vendus (Top 5) avec nombre de ventes et revenus.
- Bilan des dépenses d'approvisionnement sur la période sélectionnée.
- Export des rapports en PDF et CSV.
- Tableaux de bord spécifiques pour chaque rôle (cuisine, stock, livraison).

3.7.2 Difficultés rencontrées et solutions

Tableau 12: Difficultés module tableau de bord et rapports financiers

Difficulté rencontrée	Solution adoptée
Les requêtes d'agrégation pour le dashboard impliquaient de multiples jointures et étaient lentes sur de grands volumes.	Utilisation de <code>select_related()</code> et <code>prefetch_related()</code> sur les querysets. Ajout d'index sur <code>date_commande</code> et <code>statut</code> .
Les exports CSV de grands volumes provoquaient des timeouts.	Utilisation de <code>StreamingHttpResponse</code> avec un générateur Python pour écrire les lignes CSV progressivement.

3.8. MODULE DE GESTION DES LIVRAISONS

3.8.1 Fonctionnalités réalisées

- Gestion du parc de véhicules : liste, disponibilité, type (moto, voiture).

3.9. DROITS D'ACCÈS PAR RÔLE (FONCTIONNEMENT RÉEL DE L'APPLICATION)

Voici comment l'application est accessible selon le rôle de l'utilisateur connecté. Chaque rôle voit uniquement les interfaces qui le concernent.

Administrateur

- L'administrateur a accès à toutes les interfaces de l'application. Depuis son tableau de bord, il peut accéder à :
- Caisse : voir toutes les commandes localement.
- Cuisine : afficher les repas en attente et en cours.
- Tableau de bord général (T.B).
- Produits : gérer l'historique des commandes.
- Commandes : gérer l'inventaire en temps réel et les alertes.
- Stock : gérer le stock et les approvisionnements.
- Équipe : gérer les cuisiniers, créer des comptes employés, attribuer des rôles c'est la gestion des Ressources Humaines (RH).
- Paie : gérer et générer les fiches de paie (incomplet).
- Finance : gérer uniquement les transactions financières.
- Produits : gérer le catalogue des produits.
- Opération : gérer les équipements.
- Admin : interface d'administration directe avec la base de données.

Cuisinier

Le cuisinier a accès à un ensemble limité d'interfaces qui concernent uniquement son travail en cuisine :

- Cuisine : voir et gérer les plats à préparer.
- Produit : consulter les fiches recettes.
- Stock : consulter les quantités disponibles.
- Opération : interface de gestion de ses tâches.

Serveur

Le serveur travaille en salle. Il accède à :

- Caisse : accès à la caisse pour les paiements.
- Commandes : créer et gérer les commandes des clients.
- Produit : il ne peut rien modifier — consultation uniquement.
- Finance : il ne peut rien modifier — visualisation uniquement.
- Opération : interface opérationnelle.

Gestionnaire de Stock

Le gestionnaire de stock supervise les approvisionnements et l'inventaire :

- Produit : pour voir les produits disponibles.
- Opération : gestion des opérations (incomplet).
- Stock : gestion du stock on ne peut pas encore ajouter les ingrédients (incomplet).

Chef Cuisinier

Le chef cuisinier a accès aux mêmes interfaces que le cuisinier, avec en plus la possibilité de créer et modifier les recettes.

PARTIE IV : BIBLIOTHÈQUES UTILISÉES

Pour développer notre application, nous avons installé plusieurs bibliothèques Python via la commande pip. Voici la liste exacte des dépendances présentes dans notre fichier requirements.txt:

4.1. DJANGO (VERSION 5.2.14)

Django est le framework web sur lequel repose toute notre application. Sans lui, le projet n'existerait tout simplement pas. C'est lui qui nous a fourni la structure complète : les URL pour naviguer entre les pages, les vues pour traiter les requêtes, les modèles pour interagir avec la base de données MySQL sans écrire de SQL brut, et les formulaires pour valider les saisies des utilisateurs. Django gère aussi la sécurité : protection contre les injections SQL, protection CSRF sur les formulaires, hachage des mots de passe et gestion des sessions. Il inclut aussi une interface d'administration que nous avons personnalisée pour gérer les employés, les produits et les commandes. En résumé, Django est le pilier central qui fait le lien entre notre base de données MySQL et les pages HTML affichées à l'utilisateur.

4.2. PyMySQL (version 1.2.0)

PyMySQL est le connecteur qui permet à Django de communiquer avec notre base de données MySQL. Quand Django a besoin de lire ou d'enregistrer des données — par exemple créer une commande, consulter le stock ou récupérer la liste des employés — il passe par PyMySQL, qui traduit ces demandes en requêtes SQL que MySQL comprend. Sans PyMySQL, Django ne saurait pas comment parler à notre base de données. Nous l'avons configuré dans le fichier settings.py en définissant le moteur de base de données comme mysql et en renseignant l'hôte, le port et le nom de la base Restaurant.

4.3. Pillow (version 12.2.0)

Pillow est la bibliothèque de traitement d'images que nous avons utilisée pour gérer les photos des plats dans le catalogue de l'application. Quand un administrateur upload la photo d'un

plat, Pillow intervient automatiquement pour redimensionner l'image et la compresser si elle dépasse une certaine taille, afin qu'elle ne prenne pas trop de place sur le serveur. Pillow est aussi une exigence technique de Django : dès qu'on utilise un champ ImageField dans un modèle, Pillow doit être installé obligatoirement. Sans elle, Django refuserait de démarrer si un modèle contient des champs image.

4.4. python-dotenv (version 1.2.2)

python-dotenv nous a permis de sécuriser les informations sensibles de notre projet. Dans une application web, il y a des données qu'on ne doit jamais écrire directement dans le code : le mot de passe de la base de données, la clé secrète Django ou encore les paramètres de connexion. Si on les écrivait dans le code source et qu'on partageait le projet sur GitHub, n'importe qui pourrait les lire. python-dotenv nous a permis de les stocker dans un fichier séparé appelé .env qui n'est pas partagé sur GitHub, et de les charger automatiquement au démarrage du serveur. C'est une bonne pratique que nous avons adoptée depuis le début du projet.

5.5. asgiref (version 3.11.1)

asgiref est une bibliothèque installée automatiquement avec Django, sans que nous ayons eu à l'ajouter manuellement. Nous ne l'avons pas utilisée directement dans notre code, mais Django en a besoin en interne pour gérer la communication entre le serveur web et l'application Python. Elle permet à Django de fonctionner en mode synchrone comme en mode asynchrone, ce qui améliore les performances pour gérer plusieurs utilisateurs en même temps. On peut la voir comme un outil de fond, invisible mais indispensable au bon fonctionnement du serveur Django.

4.6. sqlparse (version 0.5.5)

sqlparse est également une dépendance installée automatiquement avec Django. Son rôle est de lire, analyser et formater les requêtes SQL générées par Django. Elle nous a été particulièrement utile pendant les phases de développement et de débogage : quand on active le mode DEBUG, sqlparse formate les requêtes SQL de façon lisible dans les logs, ce qui nous a aidés

à comprendre ce que Django envoyait réellement à MySQL et à identifier les requêtes trop lentes. Sans elle, les requêtes SQL s'afficheraient en une seule ligne illisible difficile à corriger.

4.7. tzdata (version 2026.2)

tzdata est la bibliothèque qui fournit à Python la base de données des fuseaux horaires du monde entier. Dans notre application, toutes les actions sont horodatées : la création d'une commande, le départ d'une livraison, l'heure de connexion d'un employé ou un mouvement de stock. Pour que ces heures soient correctes et affichées selon le fuseau horaire de Yaoundé (Africa/Douala, UTC+1), Django a besoin de tzdata. Sans elle, les heures enregistrées seraient décalées par rapport à l'heure locale du restaurant, ce qui rendrait l'historique des commandes et des livraisons incohérent.

Le fichier requirements.txt ne contient que les bibliothèques installées explicitement par notre équipe. Les composants internes de Django (django.contrib.auth, django.db.models, django.forms, etc.) font partie du framework lui-même et sont disponibles dès que Django est installé. Ils n'apparaissent pas dans requirements.txt, mais nous les avons largement utilisés dans l'implémentation de tous les modules.

PARTIE V : TESTS ET VALIDATION

5.1 Tests unitaires

Nous avons réalisé des tests unitaires sur les fonctions critiques de l'application pour nous assurer qu'elles produisent les bons résultats de façon isolée.

- Calcul automatique du montant total d'une commande.
- Vérification du déclenchement des alertes de seuil critique dans le module stock.
- Validation des formulaires de création de recette (champs obligatoires, quantités négatives).
- Calcul de la masse salariale mensuelle à partir des salaires des employés actifs.

5.2 Tests d'intégration

Nous avons vérifié les flux complets qui impliquent plusieurs modules :

- Flux commande complète : création → préparation cuisine → décrémentation stock → génération facture PDF ; Incomplet.
- Flux livraison : commande prête → assignation livreur → confirmation → notification client. ; Incomplet.
- Flux approvisionnement : alerte stock critique → commande fournisseur → incrémentation stock ; Incomplet.
- Flux paie : consultation employés → calcul masse salariale → génération facture. ✓ Validé.

5.3 Difficultés rencontrées lors des tests

Tableau 13: Difficultés rencontrées lors des tests

Difficulté	Solution
Les tests Django créent une base en mémoire différente de MySQL, ce qui ne permettait pas de tester les contraintes MySQL spécifiques.	Configuration d'une base de test MySQL dédiée (restaurant_test_db) dans settings.py.

CONCLUSION

Ce projet de développement d'une application web de gestion de restaurant sous Django nous a permis de mettre en pratique de façon concrète les principes du Génie Logiciel étudiés cette année.

À travers l'implémentation des modules du cahier des charges gestion des utilisateurs, des recettes, des produits, des commandes, des stocks, des ressources humaines, du tableau de bord et des livraisons notre groupe a acquis une maîtrise pratique de l'écosystème Django, de l'ORM Python/MySQL, ainsi que des bonnes pratiques de sécurité web (CSRF, RBAC, hachage des mots de passe).

Les difficultés rencontrées gestion des transactions atomiques pour la cohérence des stocks, optimisation des requêtes SQL pour le tableau de bord, gestion des droits d'accès par rôle — nous ont conduits à chercher des solutions concrètes et à mieux comprendre pourquoi Django est conçu de cette façon.

Ce projet n'est pas seulement un exercice académique. Il nous a montré comment un système d'information réel se conçoit et se développe, de la modélisation UML jusqu'à l'application web fonctionnelle. C'est une expérience que nous n'aurions pas pu acquérir uniquement en cours magistral.

WEBOGRAPHIE

- Documentation officielle Django : <https://docs.djangoproject.com/fr/5.2/>
- Documentation Pillow : <https://pillow.readthedocs.io/en/stable/>
- Documentation PyMySQL : <https://pymysql.readthedocs.io/en/latest/>
- Bootstrap 5 : <https://getbootstrap.com/docs/5.3/>
- Chart.js : <https://www.chartjs.org/docs/latest/>
- python-dotenv : <https://saurabh-kumar.com/python-dotenv/>
- Tutoriel vidéo Django : <https://www.youtube.com/watch?v=NeYSBDMIVvY>
- [Youtube.com/watch?v=NeYSBDMIVvY](https://www.youtube.com/watch?v=NeYSBDMIVvY)
- <https://youtube/NeYSBDMIVvY?si=K74K6XgmaiKrUp2y>

TABLE DE MATIRE

SOMMAIRE	2
LISTE DES FIGURES.....	4
LISTE DES TABLEAUX.....	6
INTRODUCTION.....	7
Phases du projet.....	7
Phase I: préparation du projet et initiation aux setup	8
Phase II : développement du système	8
Phase III: test et integration.....	8
Partie I : installation de l’environnement de développement.....	8
1.1 INSTALLATION de Django	8
1.2. Installation de python	10
1.3. Installation de Visual Studio Code (VS Code)	10
1.4. Installation de Git et GitHub.....	11
1.5. StarUML	11
Partie II : Analyse et développement du système	12
2.1 Analyse des méthodes employées.....	12

2.2. Analyse et modélisation UML	13
2.2.1. Diagramme de cas d'utilisation	13
2.2.2. Diagramme de classe	15
2.2.3. Diagramme de séquence	17
2.2.4. Diagramme de déploiement	43
2.2.5. DIFFICULTES RENCONTREES	45
2.3. Travail effectuée par chaque membre du groupe	45
PARTIE III : IMPLÉMENTATION DES MODULES DE L'APPLICATION	47
3.1 MODULE DE GESTION DES UTILISATEURS ET DES RÔLES	47
3.1.1 Fonctionnalités réalisées	47
3.1.2 Difficultés rencontrées et solutions	48
3.2 MODULE DE GESTION DES RECETTES	48
3.2.1 Fonctionnalités réalisées	48
3.2.2 Difficultés rencontrées et solutions	49
3.3 MODULE DE GESTION DES PRODUITS ET DU CATALOGUE	49
3.3.1 Fonctionnalités réalisées	49
3.3.2 Difficultés rencontrées et solutions	50
3.4 MODULE DE GESTION DES COMMANDES	50

3.4.1 Fonctionnalités réalisées	50
3.4.2 Difficultés rencontrées et solutions	51
3.5 MODULE DE GESTION DES STOCKS	51
3.5.1 Fonctionnalités non réalisées	51
3.6 MODULE DE GESTION DES RESSOURCES HUMAINES (RH)	52
3.6.1 Fonctionnalités réalisées	52
3.6.2 Difficultés rencontrées et solutions	52
3.7 MODULE TABLEAU DE BORD ET RAPPORTS FINANCIERS	52
3.7.1 Fonctionnalités réalisées	52
3.7.2 Difficultés rencontrées et solutions	53
3.8. MODULE DE GESTION DES LIVRAISONS	53
3.8.1 Fonctionnalités réalisées	53
3.9. DROITS D'ACCÈS PAR RÔLE (FONCTIONNEMENT RÉEL DE L'APPLICATION)	53
PARTIE IV : BIBLIOTHÈQUES UTILISÉES	56
4.1. DJANGO (version 5.2.14)	56
4.2. PyMySQL (version 1.2.0)	56
4.3. Pillow (version 12.2.0)	56
4.4. python-dotenv (version 1.2.2)	57

5.5. asgiref (version 3.11.1).....	57
4.6. sqlparse (version 0.5.5)	57
4.7. tzdata (version 2026.2).....	58
PARTIE V : TESTS ET VALIDATION	59
5.1 Tests unitaires.....	59
5.2 Tests d'intégration	59
5.3 Difficultés rencontrées lors des tests	59
CONCLUSION	60
WEBOGRAPHIE	61
TABLE DE MATIRE	62